

Synthetic image data generation using BIM and computer graphics for building scene understanding

Huaquan Ying^{a,*}, Rafael Sacks^a, Amir Degani^{a,b}

^a Faculty of Civil and Environmental Engineering, Technion-Israel Institute of Technology, Haifa, Israel

^b Technion Autonomous Systems Program (TASP), Technion-Israel Institute of Technology, Haifa, Israel

ARTICLE INFO

Keywords:

Synthetic image generation
Automatic annotation
Building information modeling (BIM)
Building scene understanding
Deep learning

ABSTRACT

Vision-based building scene understanding (BSU) is essential to many applications such as indoor navigation and 3D reconstruction. Deep learning excels in various BSU tasks but its training relies heavily on large-scale datasets, which are often burdensome and costly to obtain. This study proposes a universal and automated approach to generate synthetic building images with comprehensive annotations that support twelve BSU tasks, including camera pose estimation, scene recognition, depth estimation, 2D/3D object and object part detection, object and material semantic segmentation, object and object part instance segmentation, and panoptic segmentation. This approach captures and renders perspective views of indoor BIM model scenes to generate photorealistic images. Required annotations are computed by a ray tracing-based algorithm accelerated by two-tier axis-aligned bounding box tree indexing. The approach was validated by efficiently producing annotated synthetic images from various BIM models, and preliminary experiments demonstrated the promising efficacy of the synthetic images for deep learning model training.

1. Introduction

Understanding construction and building scenes from vision data (e.g., images, videos, and point clouds) is a prerequisite for many computer vision-based automated tasks in both construction and operation and maintenance (O&M), including construction progress and activity monitoring [1,2], site safety inspection [3], localization and navigation [4], and 3D reconstruction [5,6], to name a few. In recent years, artificial intelligence (AI), and especially the deep learning technique, dominates almost all the computer vision tasks and is increasingly used for processing construction and building scene-related vision data in the architecture, engineering, construction, and operation (AECO) industry.

Deep learning models have powerful feature representation capabilities, but their success is significantly dependent on large-scale high-quality datasets for training. Several large-scale publicly available image datasets have been created by the computer vision community, such as ImageNet [7], COCO dataset [8], and ADE20K [9]. These greatly advance the development of relevant deep learning algorithms and their real-world applications. Unfortunately, these datasets are not specific for construction and building scene understanding in the context of the AECO industry. For example, ADE20K includes >25,000 images of

building indoor and outdoor scenes annotated with over 2500 object categories. It includes many object classes irrelevant to the AECO industry and lacks sufficient object instances in many essential classes, including mechanical, electrical, and plumbing (MEP) objects in particular.

Creating a large-scale real-world dataset is often costly, laborious, and error prone. Roughly speaking, it consists of two steps: data collection and data annotation. Depending on the data source, the first step may be implemented manually by collecting the data in the real world using relevant reality-capture devices like digital cameras [10–12] or automatically through web crawling to collect data from the internet [7,13]. The manual method involves tedious labor, while the automatic method may fail to obtain sufficient valid data. For the second step, collected data is traditionally manually annotated by people. Given the large-scale size, manual data labeling can be extremely laborious and time-consuming, and often requires domain knowledge and labeling skills to ensure quality. Although many crowdsourcing services (e.g., Amazon SageMaker Data Labeling [14] and Google Cloud data labeling [15]) are available to greatly relieve the manual labor for the dataset owners, it is costly for creating large-scale datasets. For example, Amazon SageMaker Ground Truth charges \$8 to label 100 object

* Corresponding author.

E-mail addresses: enochying@campus.technion.ac.il (H. Ying), cvsacks@technion.ac.il (R. Sacks), adegani@technion.ac.il (A. Degani).

<https://doi.org/10.1016/j.autcon.2023.105016>

Received 6 December 2022; Received in revised form 16 June 2023; Accepted 5 July 2023

Available online 17 July 2023

0926-5805/© 2023 Elsevier B.V. All rights reserved.

instances for semantic segmentation [14]. In addition, the manual process can be error prone. For instance, Northcutt et al. [16] reported that the ImageNet validation set contains at least 6% label errors, including missing and incorrect labels.

Given the challenges in collecting real-world datasets, researchers have attempted to generate synthetic data from controllable and computable virtual environments automatically and cost-efficiently [17,18]. In the case of synthetic image data generation, a virtual camera is often used to capture images of virtual objects or scenes in a virtual environment such as 3D modeling and computer graphics software (e.g., Revit [19], Blender [86], and Unity [87]) [20–25]. In the virtual environment, all the elements including the virtual camera (intrinsic parameters, location, and pose), object or scene models (geometry, material, and texture), and environment (e.g., weather and illumination conditions) are editable, which enables users to capture an unlimited number of diverse images and annotate these images flexibly and automatically [26–28]. Synthetic image data has been used to train deep learning models for some computer vision and robotic tasks such as semantic segmentation of building and urban scenes [17,20], object detection for robotic manipulation [26,29], and vehicle detection [27,30]. These studies show that relevant deep learning models trained on synthetic images or real image datasets augmented with synthetic images can outperform or achieve comparable performance to models trained on real images only.

Due to the included accurate geometry and appearance (e.g., material and texture) information of building scenes, building information modeling (BIM) models have become an appropriate data source to generate synthetic images of indoor building scenes. Several studies leveraged the virtual camera of BIM authoring tools to capture synthetic images of BIM scenes and automatically annotated them with the pose of the virtual camera, such that these images can be used to train image-

based localization algorithms [21–23]. By using a Revit built-in color-coding function, Hong et al. [24] further annotated the captured BIM synthetic images with object instance masks for instance segmentation.

Although BIM has been used to generate synthetic images for several building scene understanding (BSU) tasks [e.g., 21, 22, 24], its potential has not been fully explored. Many other fundamental BSU tasks have not been investigated, such as scene recognition, depth estimation, 2D/3D object and object part detection, semantic and material segmentation, and object part segmentation (a systematic explanation of these tasks can be found in Section 2.1 below). In theory, all the annotations of BIM synthetic images required by those tasks can be acquired from BIM models. However, there is no universal approach to generate those annotations automatically. Furthermore, existing efforts to generate labeled synthetic images involve considerable manual effort and are dependent on specific BIM authoring tools. For example, Hong et al. [24]'s method to create annotated BIM images is built on Revit, thus only applicable to Revit models. Their method requires users to manually set the pose of the Revit virtual camera to capture a view and click the image export button to export the corresponding image. As mentioned earlier, it also relies on a built-in Revit function, which may not be available in other BIM authoring tools, to colorize different object types to prepare mask annotations for objects in the exported image.

The performance of deep learning models trained on synthetic images can be affected by the reality gap (i.e., differences in color and texture) between synthetic and real-world images [24,27,31,32]. In the context of BIM synthetic image generation obtaining realistic images using the built-in renderer of BIM software is challenging, even if BIM models are well-designed with fine-grained texture and material (see Figs. 1(a)–(d)). Furthermore, the rendering process is quite slow. For example, Revit's renderer takes about 40 mins (using an Intel Core i7-8700K CPU) to generate an 886×664 image (see Fig. 1(d)) in the

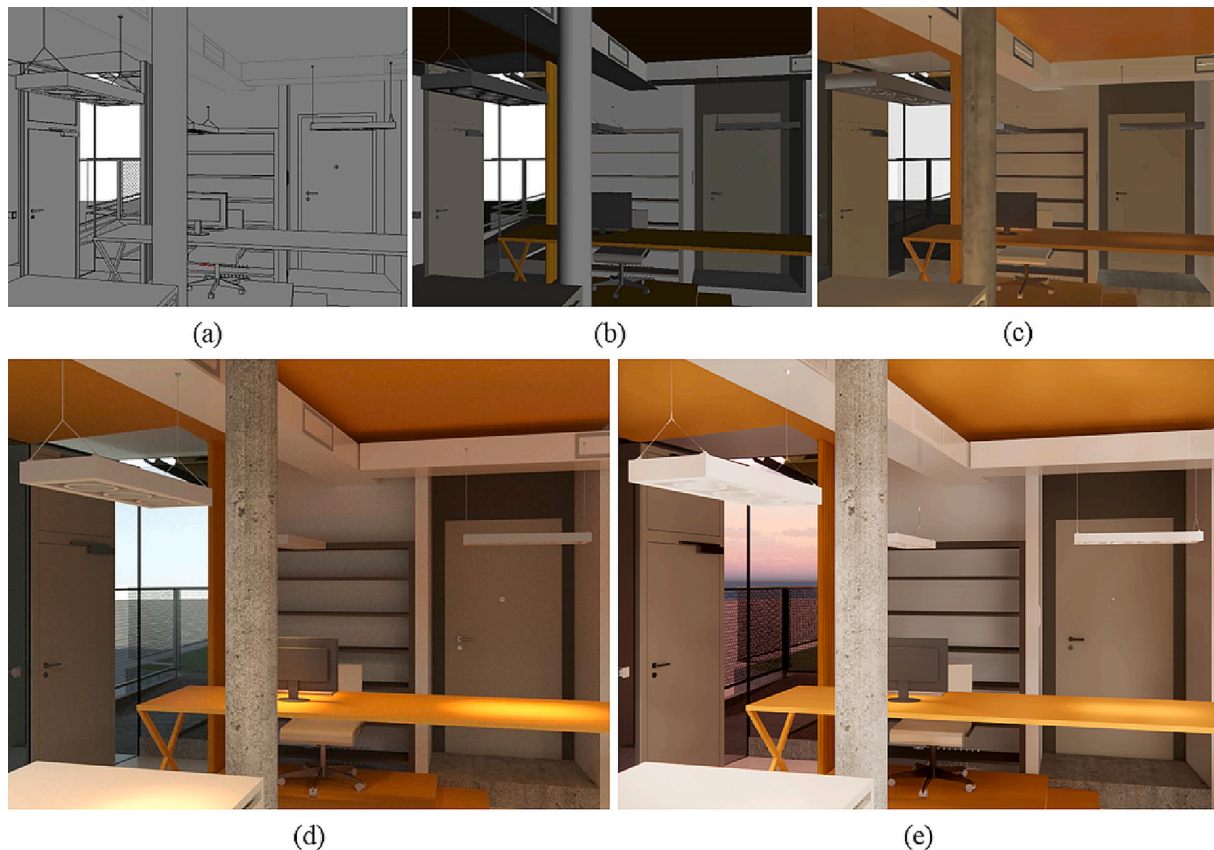


Fig. 1. BIM image generated (a) in Revit Hidden Line mode; (b) in Revit Shaded mode; (c) in Revit Realistic mode; (d) rendering in best quality mode in Revit; and (e) by Enscape in Ultra quality mode.

best quality mode. Therefore, some pioneering studies have employed domain adaptation technologies such as image-to-image translation methods to bridge the reality gap. For instance, Hong et al. [24] and Chen et al. [32] used CycleGAN [33] to transform BIM-rendered images into photorealistic images with similar appearances or styles to real-world images. However, these methods require expertise and additional non-trivial efforts, including additional data preparation for training the GAN model (see Section 2.2.3 for more details).

In fact, computer graphics has greatly matured in recent decades and can be used to generate photorealistic images at high speed [30]. For example, Enscape [34] and v-Ray [35] are advanced rendering engines that can work in many BIM software like Revit and ArchiCAD to generate photographic images in near real-time (see Fig. 1(e)). With several data transformation steps, many 3D computer graphics software like Blender and Unity can also be used to render BIM model scenes. Unfortunately, none of these computer graphics tools have been investigated to generate synthetic images of indoor BIM scenes.

To this end, this study developed a universal and automatic approach that generates photorealistic synthetic images of BIM scenes of building interiors and computes comprehensive annotations for multiple BSU tasks. The approach consists of two stages, i.e., image generation and annotation, involving five main steps, all automated: (1) determine camera poses on the space level to capture 3D perspective views using a grid-based parameterized method, (2) generate photorealistic images of captured BIM views using advanced real-time render, (3) identify valid images and add distortion effects, (4) annotate entire generated images with scene category and camera pose and each pixel of the images with the labels of depth, material type, object class, object and object part instance using a ray tracing algorithm, and (5) generate other annotations, including annotations for 2D/3D object and object part detection, based on the pixel-level labels obtained in Step (4). Except for BIM data extraction operations, the proposed approach is independent of specific BIM authoring tools.

The remainder of this paper is structured as follows. Section 2 summarizes fundamental BSU tasks in detail and reviews existing synthetic image data creation methods. Section 3 presents the technical details of our BIM and computer graphics-based synthetic image generation and annotation approach. Section 4 demonstrates the validity of the approach via a series of experiments on generating labeled synthetic images from complex BIM scenes successfully and demonstrating the effectiveness of the synthetic images for deep learning model training. Section 5 discusses the prerequisites of the approach and the limitations of the present study. Section 6 concludes this study.

2. Background

2.1. Image-based building scene understanding

BSU aims to understand a building scene by analyzing the geometric and semantic features of its contents and their relationships [36]. It includes a series of fundamental tasks on multiple perceptual levels, such as camera localization, scene recognition, depth estimation, object detection, semantic segmentation, instance segmentation, and panoptic segmentation from coarse to fine [36,37]. In what follows, the concepts of the above-mentioned BSU tasks and relevant dataset annotation requirements are discussed.

Given a single image, image-based camera localization aims to infer the pose (i.e., 3D location and 3D orientation) of the camera that captures the image. Camera localization is an essential component of many high-level applications such as robot localization and navigation and virtual/augmented reality (VR/AR). Several end-to-end deep learning-based building indoor localization approaches [22,23,31] have been developed. Data annotation to enable these approaches is to label all the training images with the 6 degrees of freedom (DOF) camera pose information.

Scene recognition aims to comprehend an entire image to understand

the scene category (e.g., “bedroom” and “living room”) that the image describes. One of the applications is robot semantic navigation in an indoor environment, which leverages semantic information about the scene for robust navigation [38]. Deep convolutional neural network (CNN) models show impressive performance on the scene recognition task [9]. Annotation for scene recognition is relatively simple by assigning a category label to each training image.

Depth estimation for an image focuses on estimating the depth of each pixel of the image. The depth of a pixel refers to the distance between the camera that captures the image and the object point in the world that corresponds to the pixel. Image depth is useful for many applications like robotic navigation and BSU tasks such as object detection and semantic segmentation [39]. The desired data annotation is to add the depth label for each pixel of the training images.

Object detection attempts to precisely estimate the class types (e.g., wall and window) and locations of objects of interest contained in an image [40]. The location of an object is generally defined by a bounding box (BB), which is a 2D rectangular box that tightly encloses the object while aligning with the edges of the image. Object part detection expands this task to a more fine-grained level by further distinguishing and localizing meaningful parts of each object (e.g., legs, seat, and backrest of a chair) [41]. For RGB-D images, the 3D object detection task can be carried out, where 3D oriented BBs (OBBs) of objects need to be estimated [42]. An OBB is a rectilinear box that tightly bounds an object or object part and aligns with its shape's orientation. Object detection has been used widely in different engineering contexts in the AECO industry, such as detecting construction equipment [13,43], construction workers [44,45], and onsite building components [46], and identifying secondary objects (e.g., MEP objects) in existing buildings [47]. To annotate an image for object and object part detection, the BBs of all the objects and object parts of interest in the image should be outlined, and for each box, the object or object part class label should be assigned.

Semantic segmentation predicts the category of every pixel in an image [48]. Therefore, to annotate an image for semantic segmentation model training, the class label of all the pixels should be given. Depending on the specific objective, the class label can be object type for object segmentation [48,49], object surface material for material segmentation [5,50], and cracks in buildings and infrastructure for crack segmentation [51,52].

Compared with semantic segmentation, instance segmentation makes finer-grained predictions by separating labels for different instances of the same class [53]. For instance, for an image containing multiple walls, semantic segmentation of walls aims to search all wall pixels, while instance segmentation further distinguishes which wall instance a searched pixel belongs to. In the AECO industry, instance segmentation has been used for precisely detecting moving objects (e.g., workers and construction vehicles) on construction sites [11] and segmenting building structural components (e.g., walls, floors, doors, and windows) [24,54]. Like object part detection, object part segmentation implements instance segmentation on the object-part level [55]. Annotating images for instance segmentation is complicated by the need to delineate the mask of each object or object part instance precisely and to assign it a class label.

Panoptic segmentation combines the tasks of semantic segmentation and instance segmentation by segmenting countable object instances (i.e., instance segmentation) and making pixelwise semantic predictions for all other amorphous areas such as glass and sky (i.e., semantic segmentation) [56]. In the context of building indoor scenes, although almost all the objects are countable things, panoptic segmentation can still be used when there are different segmentation requirements on different objects, e.g., semantic segmentation for building structural elements and instance segmentation for other objects. Image annotation for panoptic segmentation needs to distinguish objects for semantic segmentation and instance segmentation first, and then apply relevant annotation methods accordingly.

Currently, there is no building image dataset with comprehensive

annotations that can support all the above-mentioned fundamental BSU tasks. This study fills this gap by developing a universal and automatic approach to generate those annotations for synthetic images captured from BIM models.

2.2. Synthetic image data creation

Creating large-scale real-world building image requires laborious and expensive image data collection and annotation. Thanks to progress in computer graphics, generating synthetic images in a virtual environment can effectively address those challenges. Regardless of the specific objectives, synthetic image creation methods can be broadly classified into three categories: (1) direct generation from a virtual environment; (2) inserting objects of interest into real-world background images; and (3) image-to-image translation.

2.2.1. Direct generation from a virtual environment

This method captures synthetic images of 3D models of building scenes by using a virtual camera in 3D modeling and computer graphics software. As discussed above in Section 1, due to editability and computability of the elements (e.g., virtual camera, scene model, and lighting environment) in the virtual environment, theoretically an unlimited number of varied images can be captured and annotated automatically.

This method has been widely used to construct training datasets. Several studies used a virtual camera to capture images of 3D building models in Blender and to annotate the images with camera pose to train relevant deep learning models for indoor camera pose estimation [22,23]. These models show promising accuracies when tested on real images. Ha et al. [21] extracted BIM-rendered images of building indoor scenes to support an image retrieval-based camera pose estimation. Calderon et al. [57] developed a method to generate synthetic image sequences that capture simulated operations of a kinematically articulated excavator modeled in Unity. The deep learning model trained on the synthetic image sequences achieved superior performance in predicting excavator pose sequence compared to the model trained on real-world data. Similarly, Mahmood et al. [58] constructed synthetic images of virtual excavators for 3D excavator pose estimation. To make the synthetic images more realistic, this study considered natural visual features such as lighting, occlusions, image distortion, dust, and natural background. Assadzadeh et al. [28] proposed a domain randomization approach to augment the synthetic excavator images. Specifically, the approach randomizes a wide range of parameters including the view-point and field of view (FOV) of the virtual camera, lighting, floor and background textures, occluding objects, and dust to increase the effectiveness of the synthetic images for deep learning model training.

2.2.2. Inserting objects of interest into real-world background images

Although Method (1) can generate annotated synthetic images conveniently in a virtual environment, one of its main limitations is the appearance and style difference with real-world images, which can impede the performance of the resulting deep learning algorithms. As discussed earlier, several techniques including high-fidelity scene modeling and domain randomization have been introduced to relieve the reality gap. Nevertheless, these efforts may find it difficult to address the image backgrounds that are often natural (e.g., grass) or complex manmade (e.g., construction sites) scenes. Some efforts handle this issue by inserting synthetic or real-world image patches of objects of interest into real-world background images. For example, Dwibedi [59] proposed a ‘cut and paste’ method to generate synthetic images for instance segmentation. This method first cuts objects from original real-world images and then pastes the object patches into random real-world background images to produce desired synthetic images. Soltani et al. [60] adopted Method (1) to capture virtual images of a 3D excavator model and then overlaid the captured excavator images to diverse real-life construction site images. With a similar idea, Lee et al. [25]

developed a more universal method to generate synthetic images for onsite construction equipment detection. This method formalized a series of controllable parameters for capturing virtual construction equipment images as well as adjusting real-world construction site images. By randomizing these parameters, diverse images of construction equipment and sites are generated separately, which are further combined to generate desired synthetic images.

2.2.3. Image-to-image translation

This method bridges the reality gap by transforming virtual rendering images into realistic images and meanwhile maintaining the annotations of original images. Nowadays, Generative Adversarial Network (GAN)-based algorithms dominate various image-to-image translation problems [61]. Specifically, CycleGAN [33] is widely used to map images from two different domains. Hong et al. [24] adopted CycleGAN to transform BIM rendering images with real-world textures for instance segmentation of building indoor scenes. The CycleGAN model that enables the transformation was trained with a set of paired BIM images and real-world photographs that describe the same scenes in the virtual BIM model and the real world, respectively. Likewise, Chen et al. [32] trained a CycleGAN model to create a photorealistic BIM image database for image retrieval. These studies demonstrate promising results in terms of dataset generation for training relevant deep learning models. However, compared with Methods (1) and (2), this method has additional challenges. First, a large-scale dataset comprising pairs of BIM rendered images and real-life images needs to be prepared for CycleGAN model training. Second, the training itself requires considerable effort and expertise because of the well-known instability in GAN model training [62,63].

2.2.4. Summary

Method (1) is the simplest and most straightforward of the three methods because requires neither real-world image collection nor additional expensive processing. Until recently, however, the image quality that could be achieved by available rendering algorithms was such that the potential reality gap with real-world images limited its effectiveness. This is the reason why researchers developed the various techniques in Method (2) and Method (3) to augment the virtual images generated by Method (1).

With the significant progress in BIM adoption in the construction industry, high-quality BIM models of a wide variety of building types are now available and software developers have released advanced computer graphics software capable of generating photorealistic images of BIM scenes (e.g., Enscape [34], v-Ray [35], Blender [86] and Unity [87]). Given these developments, it may now be possible to generate synthetic images from renderings of virtual BIM scenes of sufficient quality for relevant deep learning model training for BSU tasks. If so, the barriers posed by tedious and inconvenient real-world image collection and/or technically complex and costly postprocessing operations (Methods 2 and 3) could be avoided and practical application of scene understanding would become feasible. That Method (1) is now feasible for certain building types is thus the hypothesis underlying this research.

3. Generation of synthetic images for building scene understanding

3.1. Overview

The objective of this study is to develop a universal and automatic approach to generate photorealistic synthetic RGB images of indoor building scenes with comprehensive annotations using BIM and computer graphics. The resulting annotated images enable various deep learning-based fundamental BSU tasks, including camera pose estimation, scene recognition, depth estimation, 2D/3D object and object part detection, object and material semantic segmentation, object and object part instance segmentation, and panoptic segmentation.

Fig. 2 illustrates the detailed workflow of the approach. It captures and annotates images of indoor BIM scenes space-by-space. The approach first extracts all the necessary data related to interior spaces of interest and physical objects from the input BIM model, which are fed to two successive stages: photorealistic BIM synthetic image creation and desired annotation generation. In each target space, the first stage automatically determines the feasible poses of a configured virtual camera to capture 3D perspective views of the model scene. These captured views are rendered into photorealistic images by using a 3D renderer, of which valid images are identified by removing duplicates and images that are incomprehensible to human. Lastly, to mimic the distortion effects in real images, which are caused by manufacturing imperfections in the camera lens, the obtained valid images are further distorted using predefined coefficients. In this stage, two annotations of each valid image, including the camera pose and scene category, can be obtained directly.

The second stage computes all the remaining annotations of each valid image progressively. Specifically, this stage first detects all the physical objects that partially or completely fall in the view frustum of the virtual camera that captures this image. Then, all the detected objects are indexed as a two-tier axis-aligned bounding box (AABB) tree to accelerate the next ray tracing process, which produces pixelwise annotations including depth, labels for object and object part instance segmentation, labels for object and material semantic segmentation, and labels for panoptic segmentation. The ray-tracing process also yields a

point cloud as a valuable by-product. The point cloud consists of the 3D object points that correspond to the object pixels in the image. Along with the obtained annotations, the point cloud is used to compute remaining annotations including 2D BBs and 3D OBBs of objects and object parts.

Note that for the BSU tasks including object semantic segmentation, 2D/3D object and object part segmentation, and object and object part instance segmentation, our approach generates fine-grained annotations on a basic object/object part concept level. Complex concept-level annotations can be derived by combining relevant fine-grained annotations. For instance, for a wall with multiple windows in an image, our approach would separately annotate the wall instance and each window instance for the object instance segmentation task. The annotation of a more complex concept – “wall with windows” can be obtained by merging the masks of the wall and all the hosted windows and labeling it as “wall with windows”.

3.2. BIM data extraction

The approach starts with extracting interior space objects and physical objects from the input BIM model to prepare the following steps. Specifically, for each space, the solid geometry (represented as a triangle mesh) and the name attribute data are retrieved for automatic camera pose determination (see Section 3.3.2) and scene category annotation (see Section 3.3.4), respectively. An entire BIM model may

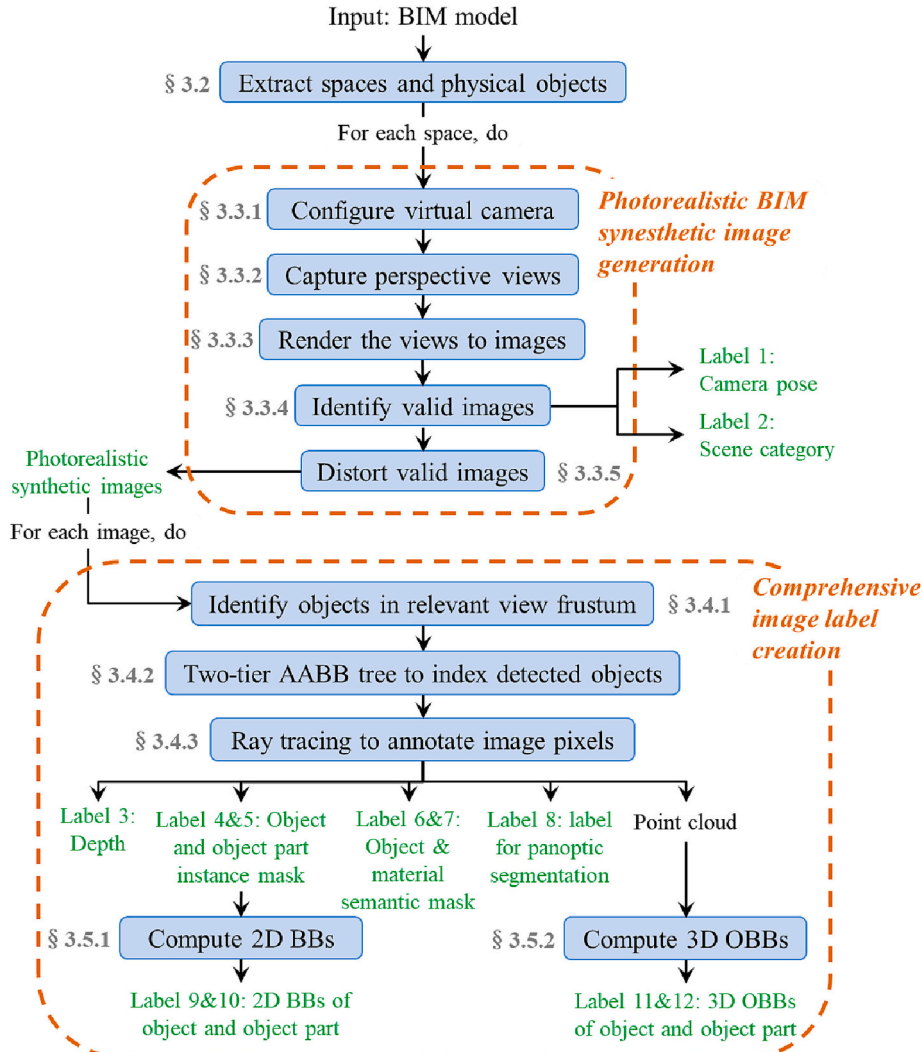


Fig. 2. Pipeline for generating labeled photorealistic images from a BIM model.

contain exterior spaces or interior spaces with similar or duplicate scenes. Distinct interior spaces are thus selected as the targeted ones. Physical objects are extracted for annotation computing since they constitute the contents of the images to be captured (see Section 3.4). A BIM object is identified as physical if it carries material and solid geometry data. For each physical object, the following data are retrieved: object class, instance ID (i.e., global ID), and triangle mesh. Each triangle in the triangle mesh is further marked with the surface material and the outward normal vector, which can be obtained by the method presented in [64]. If the object consists of multiple geometric parts, each triangle also carries the information of the part class it belongs to and the part instance ID.

All those data can be directly retrieved using the application programming interface (API) of relevant BIM authoring tools or IFC toolboxes (e.g., IFC Engine [65] and xBIM toolkit [66]), depending on the data format of the input BIM model. In addition, AABBs of the extracted spaces and physical objects are inferred from their triangle meshes and used to accelerate relevant geometric operations in downstream processes.

3.3. Photorealistic synthetic image generation

The approach leverages the virtual camera embedded in BIM authoring tools or computer graphics software to capture 3D perspective views of indoor scenes, which are then rendered and processed to create desired RGB images. First, the parameters that characterize the virtual camera are configured to set up the virtual camera (Section 3.3.1). To capture as many effective views as possible efficiently, the method to determine the location and orientation of the virtual camera for view capturing is carefully designed (Section 3.3.2). The captured views are then rendered into photorealistic images (Section 3.3.3) and their validity is further checked (Section 3.3.4). Lastly, all the valid images are distorted to provide the desired images (Section 3.3.5).

3.3.1. Virtual camera configuration

The virtual camera models and associated parameters implemented in different modeling and computer graphics software are almost the same. Fig. 3 illustrates the virtual camera model in Revit 2023. The key parameters of the camera model include focal length (f), horizontal ($hFOV$) and vertical ($vFOV$) FOV, and the distances (d_n and d_f) between the camera and the near and far planes of the so-called view frustum (VF). The VF represents the visible space to the virtual camera, i.e., only objects partially or completely in the VF can be captured by the camera. It is a truncated pyramid, whose apex refers to the origin of the camera. It can be completely determined by the eight vertices of the near and far planes, whose coordinates in the camera coordinate system (CCS) can be easily calculated based on d_n , d_f , $hFOV$, and $vFOV$. Likewise, the width

(w) and height (h) of the viewport, on which the image coordinate system (ICS) is located and all the objects in the VF are projected for image generation, can be derived from f , $hFOV$, and $vFOV$. Furthermore, the parameters related to the size of output images, including width (W) and height (H) in pixel, need to be specified.

Note that, images captured by a real digital camera may have radial and tangential distortions caused by the manufacturing imperfections of the camera lens. However, a virtual camera is often an ideal pinhole camera model without lenses, thus lacking the ability of simulating distortion effects directly. Mathematically, both radial and tangential distortions can be described by two independent parameters (denoted (k_1, k_2) and (p_1, p_2)). Hence, to enhance the realism of the virtual camera, the four distortion parameters are introduced. Synthetic images with distortion effects can then be obtained by distorting the ideal images captured by the virtual camera based on the distortion parameters (see Section 3.3.5).

In summary, by customizing the group of parameters including f , ($hFOV$, $vFOV$), (d_n , d_f), (k_1 , k_2 , p_1 , p_2), and (W , H), virtual cameras in various specifications can be set up to capture images in diverse forms on the aspects of FOV, resolution, size, and quality. We note that the axes directions of the three coordinate systems (i.e., CCS, ICS, and pixel coordinate system (PCS)) involved in the virtual camera model may vary in different software; hereafter, this study uses the settings of Revit (see Fig. 3) to demonstrate the proposed approach.

3.3.2. Perspective model view capture

After the virtual camera is configured, the next step is to determine its potential poses in each target space such that corresponding 3D perspective views can be captured using the configured virtual camera. A camera pose consists of two components: location (i.e., the origin of the CCS) and orientation (i.e., the axis directions of the CCS), which can be represented by a 4×4 transformation matrix ($TM_{CCS \rightarrow MCS}$) from the CCS to the MCS, as shown in Fig. 3. Herein, a two-step parameterized method is developed to efficiently sample feasible camera poses in the space, as illustrated in Fig. 4.

First, the feasible locations of the virtual camera in a space are sampled using a grid. To simplify the computation, an AAB of the space is used as the initial feasible region. Then, the AAB is subdivided as a 3D grid along the three axes of the MCS. The grid corner points refer to the initial set of camera locations in the space. Given the minimum vertex ($x_{min}, y_{min}, z_{min}$), length (l_x), width (w_x), and height (h_x) of the space AAB, and horizontal (δ_x and δ_y) and vertical (δ_z) grid interval, the coordinates (x_l, y_l, z_l) of a grid corner can be represented by:

$$\begin{cases} x_l = x_{min} + i\delta_x \\ y_l = y_{min} + j\delta_y \\ z_l = z_{min} + k\delta_z \end{cases} \quad (1)$$

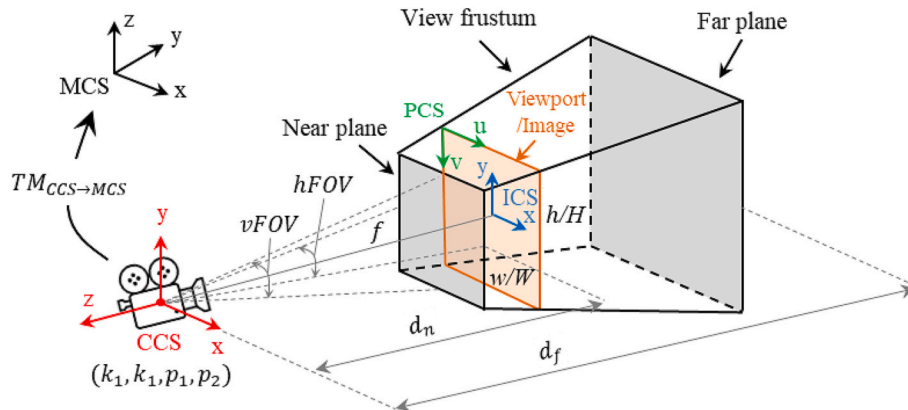


Fig. 3. Virtual camera model in Revit improved by considering distortion effects. Note: In Revit, the viewport is an independent plane slightly behind the near plane while in many other software, the viewport refers to the near plane.

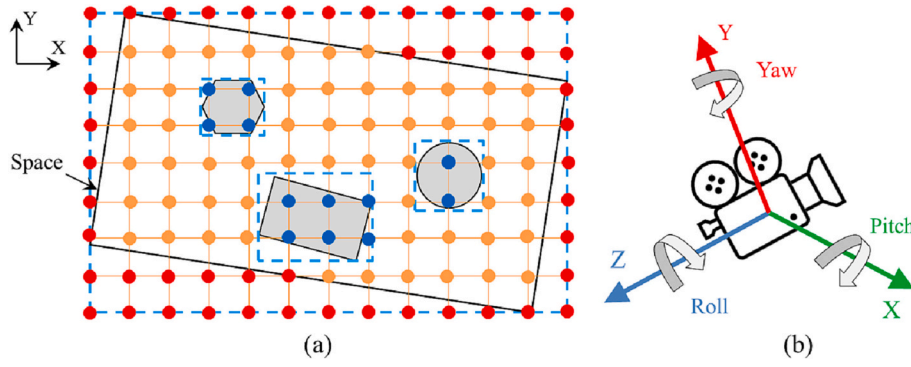


Fig. 4. Method to determine feasible locations (a) and orientations (b) in a space.

where i, j , and k refer to an integer in $[0, l_s/\delta_x]$, $[0, w_s/\delta_y]$, and $[0, h_s/\delta_z]$, respectively. Since i, j , and k are independent parameters, around $l_s w_s h_s / (\delta_x \delta_y \delta_z)$ initial camera locations are obtained for the space.

Some grid corners may fall outside the space (see the red points in Fig. 4(a)), and these should be excluded. Point-in-Polyhedron tests [67] are implemented to find all the grid corners (see the brown and blue points in Fig. 4(a)) located in the exact space geometry – a triangle mesh-based polyhedron.

The set of grid corners obtained so far is further narrowed down by removing corners (see blue points in Fig. 4(a)) inside any physical objects in the space. First, for efficiency, those physical objects are identified from the set of all extracted physical objects whose AABB is contained by or intersects the space's AABB based on the AABB-AABB intersection test [67]. Objects that enclose spaces (e.g., walls, floors, ceilings, windows, and doors) are directly excluded. Then, grid corner points inside the AABBs of the identified objects can be easily filtered. The remaining grid corners (see brown points in Fig. 4(a)) refer to the desired locations of the virtual camera. Since the virtual camera is volumetric, in the practical implementation of the above-described process, we shrink the space's AABB by 200 mm and expanded the involved physical objects' AABBs by 100 mm in all three axis directions. This eliminates unreasonable camera locations where the virtual camera intersects the space's boundaries or physical objects.

Once the virtual camera's locations are determined, its orientations on each location need to be set up. The camera orientation in the MCS has 3DOF, which can be defined by three rotation angles (yaw – α , pitch – β , and roll – γ) along the three axes of the MCS, respectively (see Fig. 4(b)). The rotation matrix ($\mathbf{R}$) from the MCS to the CCS can be constructed by

$$\mathbf{R} = \begin{bmatrix} \cos\alpha\cos\beta & \cos\alpha\sin\beta & -\sin\alpha\cos\beta & \cos\alpha\sin\beta & \sin\alpha\sin\beta \\ \sin\alpha\cos\beta & \sin\alpha\sin\beta & \cos\alpha\cos\beta & \sin\alpha\sin\beta & -\cos\alpha\sin\beta \\ -\sin\alpha & \cos\alpha\sin\beta & \cos\alpha\cos\beta & \sin\alpha\sin\beta & \cos\alpha\cos\beta \end{bmatrix} \quad (2)$$

where, α, β , and γ belong to $[0, 2\pi)$ theoretically. Angular resolution parameters $\varphi_\alpha, \varphi_\beta$, and φ_γ are introduced to discretize the three angles in their feasible regions by

$$\begin{bmatrix} \alpha \\ \beta \\ \gamma \end{bmatrix} = \begin{bmatrix} u\varphi_\alpha \\ v\varphi_\beta \\ w\varphi_\gamma \end{bmatrix} \quad (3)$$

where, u, v , and w belong to an integer in $[0, 2\pi/\varphi_\alpha)$, $[0, 2\pi/\varphi_\beta)$, $[0, 2\pi/\varphi_\gamma)$. Around $8\pi^3/(\varphi_\alpha\varphi_\beta\varphi_\gamma)$ camera orientations can be obtained for each location.

Each of these camera orientations corresponds to an image to be captured, which will be annotated by the downstream expensive image labeling processes. Some images can be obtained by rotating other images (denoted as base images) by multiples of 90 degrees (for square images with an aspect ratio of 1:1) or 180 degrees (for rectangular im-

ages), as illustrated in Fig. 5. Those images thus do not need to be captured and annotated by the downstream processes but can be attained by rotating relevant base images and their annotations appropriately, which is much cheaper computationally. The above situations happen when fixing α and β and changing γ (i.e., rotating the camera along the z-axis of the CCS) (see Fig. 5). Hence, the feasible region of γ in Eq. (2) can be reduced to $[0, \pi/2)$ (for square images) or $[0, \pi)$ (for rectangular images). Similarly, when fixing γ , changing β by 180 degrees would generate two sets of images that are vertically flipped to one another, which means the feasible region β can be reduced to $[0, \pi)$. Therefore, w is restricted to an integer in $[0, \pi/(2\varphi_\alpha))$ or $[0, \pi/\varphi_\alpha)$ and v in $[0, \pi/\varphi_\beta)$. The number of images for a specific location to be captured and labeled is reduced by up to 8 times to $\pi^3/(\varphi_\alpha\varphi_\beta\varphi_\gamma)$.

For specific $hFOV$ and $vFOV$, the values of φ_α and φ_β determine the overlap between two adjacent images captured along y-axis (i.e., rotating α) and x-axis (i.e., rotating β) of the CCS, respectively. The overlap ratio to $hFOV$ (O_h) and $vFOV$ (O_v) can be computed by

$$\begin{cases} O_h = \begin{cases} (hFOV - \varphi_\alpha)/hFOV & \text{if } \varphi_\alpha < hFOV \\ 0 & \text{others} \end{cases} \\ O_v = \begin{cases} (vFOV - \varphi_\beta)/vFOV & \text{if } \varphi_\beta < vFOV \\ 0 & \text{others} \end{cases} \end{cases} \quad (4)$$

In this study, to eliminate capture of duplicate scenes, O_h and O_v are set to zero, which means that φ_α and φ_β are set to be not less than $hFOV$ and $vFOV$ respectively. φ_γ is suggested to be 45 degrees, which provides sufficient difference between two images captured by rotating γ .

Based on the set camera location ($\mathbf{L} = (x_l, y_l, z_l)$) and orientation ($\mathbf{R}$), the camera pose represented in form of $TM_{CCS \rightarrow MCS}$ is

$$TM_{CCS \rightarrow MCS} = \begin{bmatrix} \mathbf{R}^{-1} & \mathbf{L}^T \\ 0 & 1 \end{bmatrix} \quad (5)$$

3.3.3. View rendering

The captured indoor perspective views are rendered to generate synthetic images. The render effects of a BIM view are mainly decided by three aspects:

- (1) Physical model of the view, which contains the geometry, material, and texture data of the included objects. The optical properties of objects used for rendering such as reflectivity, color, and transparency/absorption are incorporated in the textures and material type properties of the objects.
- (2) Indoor illumination, which results from artificial lighting and natural illumination. Artificial lighting is determined by lighting fixtures defined in the BIM model, which specify all light-related properties such as location, shape and brightness of light source, light distribution (e.g., spherical, hemispherical, or spot), and photometric characteristics (e.g., light loss, intensity, and color).

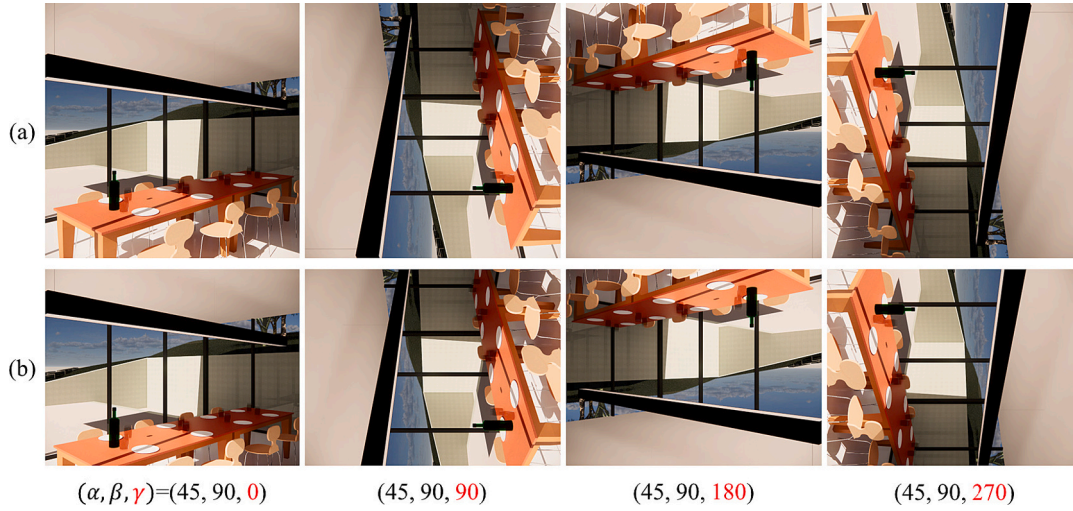


Fig. 5. Square (a) and rectangular (b) images captured at the same location with different orientations (α and β are fixed, γ varies by 90 degrees). For square images, the last three images can be obtained by rotating the first image by angles of 90, 180, and 270 degrees respectively. For rectangular images, the third and the fourth images can be obtained by rotating each of the first and the third images by 180 degrees.

Natural illumination is related to sunshine, which is determined by the geolocation of the building, the weather, and the date and time. Except for the date and time that can be selected flexibly, all the parameters that define the lighting conditions could be fixed in the BIM model.

- (3) The rendering engine used for rendering the view. For the same BIM scene and the same indoor illumination setting, different rendering engines may generate aesthetically different images because they use different algorithms and techniques to simulate the behavior of light and its interaction with the objects in the scene. The aesthetic differences caused by different rendering tools are beyond the scope of this study. We adopted a rendering software called Enscape, which can run directly in various BIM environments, to generate photorealistic images of captured BIM views in near real-time. By synchronizing captured BIM views, Enscape would extract all the above-mentioned data associated with these views automatically for rendering.

To generate synthetic images with diverse visual effects while maintaining realism, we keep all the data except the date and time as defined in the BIM model and randomize the date and time for captured views. More specifically, for a view, a random date in the year 2021 is assigned, and a time (10:00 am, 15:00 pm, or 22:00 pm) is randomly picked. We do not randomize the year because for a specific location, different years shall have a similar sun path on the same day. We avoid randomizing the time on the hour or minute level because the visual differences caused by sun location differences between minor time

intervals are often minimal. For the same view, the images rendered at a different date and/or time could have a significantly different visual appearance, as shown in Fig. 6. This effectively adds visual diversity to the resulting synthetic images. In addition, on top of the synthetic images, many other image visual effects such as noise (e.g., blurring), brightness adjustment, scaling, and flipping can be introduced to create new images by using relevant image argumentation techniques. These are outside the scope of this study.

3.3.4. Valid image identification

The grid-based BIM view capture method provides a universal means to generate geometrically feasible camera poses in an indoor space. However, it may result in “invalid” images for dataset creation in two cases: (1) images quite similar or same to other images (see Fig. 7 (b)); and (2) images lacking sufficient geometric features that make incomprehensible even to a human observer (see Fig. 7(c)). These images need to be identified and discarded since they are useless for deep learning model training but increase the computing burden.

Regarding the first case, a similar checking is implemented to all the rendered images of a space to identify and remove similar or duplicate images. Specifically, the cosine similarity ($\cos$) between two images is used to measure their similarity, which can be calculated by

$$\cos = \frac{\mathbf{ima}_1 \bullet \mathbf{ima}_2}{\|\mathbf{ima}_1\| \times \|\mathbf{ima}_2\|} \quad (6)$$

where, $\mathbf{ima}_1$ and $\mathbf{ima}_2$ refer to a flattened (i.e., flattening 3D image array

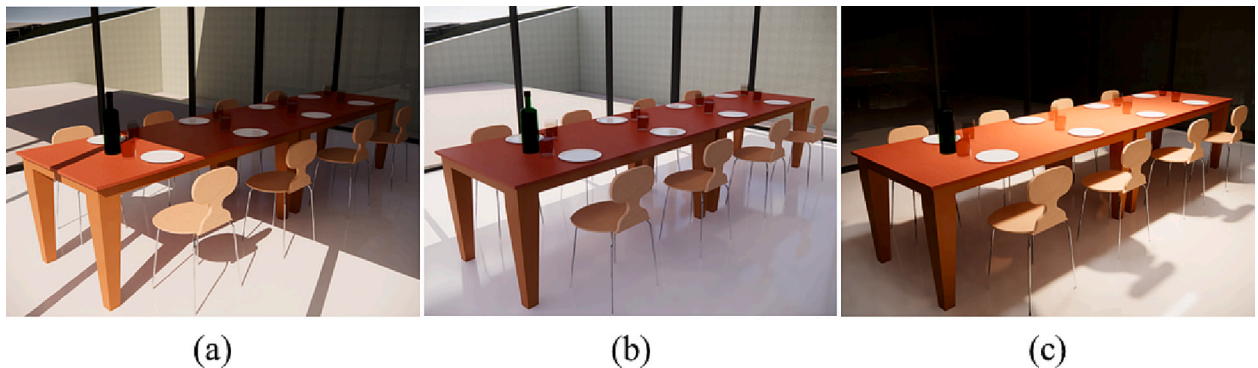


Fig. 6. Rendering images of a BIM view set with different times: (a) 10:00 am; (b) 3:00 pm; and (c) 10:00 pm.

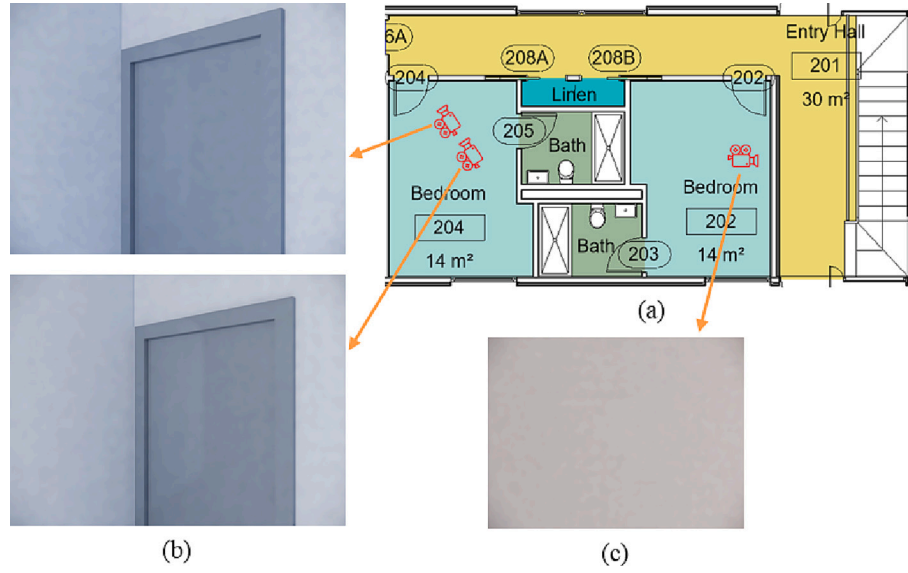


Fig. 7. Cases of “invalid” images: (a) camera locations; (b) two similar images; and (c) an image with undistinguishable contents.

into 1D vector) and normalized (i.e., dividing pixel color values by 255) vector of the two target images. If $\cos$ is larger than a threshold, the two images are confirmed as similar and any one of them needs to be removed. The criterion of image similarity becomes stricter as the threshold value increases. To ensure that only the images with a high degree of similarity are identified as invalid and removed, a relatively high value of 0.99 is set. This avoids unnecessary discarding of images with slight similarities, thereby preserving potentially valuable training data for deep learning model development. For large-sized images, both can be downsampled by the same ratio before calculating the similarity for efficiency.

For the second case, captured images with insufficient edges, e.g., fewer than two edges, are recognized as invalid. To detect the edges in an image, the image is converted into a gray image first, which is further noised by gaussian blurring for better edge detection. Finally, the Canny edge detection algorithm [68] is applied to the blurred gray image. Fig. 8 shows a valid and an invalid sample image and their detected edges.

For each valid image, the annotation of the known camera pose is described by the 3D location (x, y, z) and the three rotation angles (i.e., yaw, pitch, and roll) in the MCS. Similarly, since the space where the image is taken is known, the image can also be straightforwardly labeled with the scene category, i.e., the name of the space, which has been extracted in Section 3.2.

3.3.5. Valid image distortion

As discussed in Section 3.3.1, compared to a real camera, the virtual camera is ideal without any optical errors in lens. The generated rendering images thus do not have corresponding distortions. This step

distorts the valid images based on the predefined distortion coefficients (i.e., k_1, k_2, p_1, p_2) of the virtual camera to make the images more realistic.

For each valid image, the basic idea of distortion is to update the values (including alpha, red, green, and blue) of every pixel (u, v) with that of the corresponding pixel (u_d, v_d) in the original image, which can be calculated by:

$$\begin{cases} u_d = f_x \left((1 + k_1 r^2 + k_2 r^4)x + 2p_1 xy + p_2 (2x^2 + r^4) \right) + c_x \\ v_d = f_y \left((1 + k_1 r^2 + k_2 r^4)y + 2p_2 xy + p_1 (2y^2 + r^4) \right) + c_y \end{cases} \quad (7)$$

where (x, y) refer to the coordinates of the point corresponding to pixel (u, v) in the CCS, r is the Euclidean distance between the point (x, y) and the origin of the CCS, and (f_x, f_y, c_x, c_y) are the intrinsic parameters of the virtual camera that captures the image. All these parameters can be obtained by

$$\begin{cases} f_x = -fW/w \\ f_y = fH/h \\ c_x = W/2 \\ c_y = H/2 \end{cases} \quad (8)$$

$$\begin{cases} x = (u - c_x)/f_x \\ y = (v - c_y)/f_y \\ r = \sqrt{x^2 + y^2} \end{cases} \quad (9)$$

where, f, W, H, w , and h refer to the virtual camera's focal length, width, and height of the image in pixel, and the actual width and height of the camera's viewport. They are either preset or can be calculated from other fundamental parameters of the virtual camera, as explained in



Fig. 8. (a) Valid image and detected edges, and (b) invalid image and detected edges.

Section 3.3.1.

Note that (u_d, v_d) calculated by Eq. (7) may be not integers, in which case the associated pixel values cannot be found. To tackle this issue, a bilinear interpolation is implemented to derive the pixel values by weighting the values of the four nearest pixels of (u_d, v_d) .

3.4. Pixelwise annotation generation

So far, valid photorealistic synthetic images have been generated, and two types of annotation – camera pose and scene category – have been obtained for each image. In this section, we compute pixelwise annotations including depth, object and object part instance labels, object and material labels for semantic segmentation, and labels for panoptic segmentation. The basic idea is to compute the 3D object point that each image pixel corresponds to using a ray tracing algorithm (Section 3.4.3). The rays that originate from the camera through each pixel are traced against physical BIM objects to compute relevant object points, from which the required annotations can be derived. Given the large number of pixels in an image and numerous objects in the BIM model, two tactics are introduced to accelerate the intensive ray tracing process: reduce the scope of BIM objects to be checked for each ray (Section 3.4.1) and spatially index these objects for faster queries of each ray (Section 3.4.2).

3.4.1. Identify BIM objects in the view frustum

Only physical objects fully or partially located in the VF can appear in a virtual camera image. Hence, only these objects are considered in the ray tracing process. We identify such objects indirectly by excluding objects disjoint with the VF from the set of objects we seek. Fig. 9 illustrates all the potential topological relationships between extracted BIM objects and the VF. Using an algorithm based on the *Disjoint* topological query operator developed by [69], all the objects apart from the VF can be searched. Note that the operator *Disjoint* works on the triangle mesh representation of both operand objects. The triangle meshes of BIM objects have been prepared in Section 3.2. The triangle mesh of the VF can be easily obtained because all its six faces are simple quadrangles. Considering that a full-scale BIM model often contains numerous objects, before the exact *Disjoint* query, a simple and fast AABB-AABB intersection test is implemented for pre-checking. BIM objects whose AABB disjoins the VF's AABB are excluded.

3.4.2. Index identified BIM objects

The identified BIM objects in the VF are spatially indexed to further expedite the subsequent ray tracing process. Existing studies have demonstrated that spatial indexing can induce significant efficiency

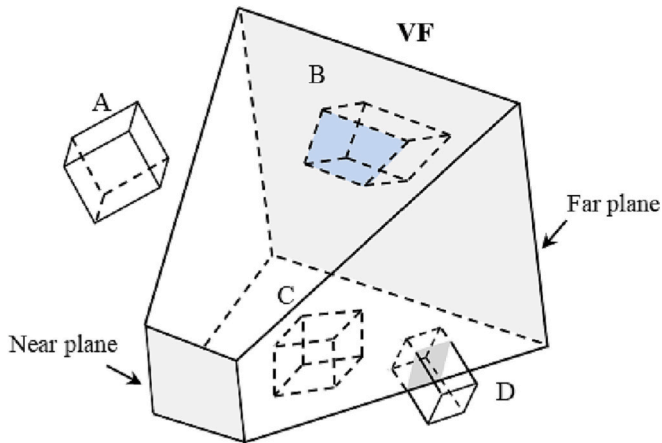


Fig. 9. Potential topological relationships between BIM objects and the VF: A – disjoint; B – touch; C – inside; D – intersect. B, C, and D are objects of interest for downstream processes.

improvement for various computational geometry algorithms in the context of BIM [69–72]. This study follows Ying and Lee [70]’s idea to establish a two-tier AABB tree indexing strategy. An AABB tree is a special bounding volume hierarchy (BVH) that indexes geometric primitives (e.g., points, triangles, and polyhedrons) into a binary tree based on their AABBs. AABB trees are used in this study because: (1) they are fast and easy to build compared to other spatial indexing structures like R-trees; (2) they have low memory overhead and enable efficient storage and retrieval of BIM object information; and (3) they are well-suited for ray tracing because they allow for quick rejection of objects that do not intersect with a given ray. Fig. 10 illustrates the two-tier indexing strategy in 2D. The first-tier indexing organizes all the identified objects into an AABB tree (denoted ‘model tree’) based on their AABBs. The second-tier indexing creates an individual AABB tree (denoted ‘object tree’) for each object based on its triangle mesh. Please see [70] for more implementation details of the two-tier AABB tree.

We implemented two common tree node splitting methods for the tree construction: a middle point-based method and surface area heuristic (SAH)-based method [73]. The former is fast for tree construction but may result in an unbalanced and spatially sparse tree, which is a disadvantage when traversing the tree. SAH takes more time to implement but generally produces an optimized tree that can significantly benefit tree traversal operations. SAH is recommended only when the target BIM scenes and the image sizes are extremely large.

3.4.3. Ray tracing to label each image pixel

The goal of the ray tracing is to compute the corresponding 3D object point of each image pixel in the BIM model scene. Information associated with the object point, including coordinates, object and/or object part class and instance ID, and material, can be obtained in this way to generate image pixelwise annotations. The camera’s origin (i.e., the CCS’s origin), the point (denoted as pixel point) on the viewport that corresponds to the pixel, and the desired 3D object point are colinear, as shown in Fig. 11. The 3D object point is the point of intersection of the ray emitted from the camera through the pixel point with a BIM object surface in the VF that lies closest to the camera. The process has two steps: ray construction and ray tracing.

For a pixel, the corresponding ray is constructed in the MCS by taking the camera’s origin as the starting point, and the directional vector from the camera’s origin to the pixel point as the ray’s direction. The camera’s origin (x_c, y_c, z_c) in the MCS is known (see Section 3.3.1). The coordinates of the pixel point in the MCS are needed. Given the camera parameters including focus (f), the width (w) and the height (h) of the viewport, and the width (W) and the height (H) of the image, and the known spatial relationships between CCS, ICS, and PCS (assuming they are consistent with those in Fig. 11), the pixel point $(x_{p-c}, y_{p-c}, z_{p-c})$ in the CCS corresponding to the image pixel (u, v) can be calculated by:

$$\begin{cases} x_{p-c} = \frac{w}{2} - \frac{wu_d}{W} \\ y_{p-c} = \frac{hv_d}{H} - \frac{h}{2} \\ z_{p-c} = -f \end{cases} \quad (10)$$

Note that distorted pixel coordinates (u_d, v_d) obtained by Eq. (7) rather than the original pixel coordinates (u, v) are used in Eq. (10) to compute the annotations of distorted images. Then the pixel point is transformed into the MCS where BIM scenes are defined by

$$\begin{bmatrix} x_p \\ y_p \\ z_p \\ 1 \end{bmatrix} = TM_{CCS \rightarrow MCS} \bullet \begin{bmatrix} x_{p-c} \\ y_{p-c} \\ z_{p-c} \\ 1 \end{bmatrix} \quad (11)$$

where $TM_{CCS \rightarrow MCS}$ refers to the transformation matrix from the CCS to the MCS, which was obtained in Section 3.3.2, and (x_p, y_p, z_p) the

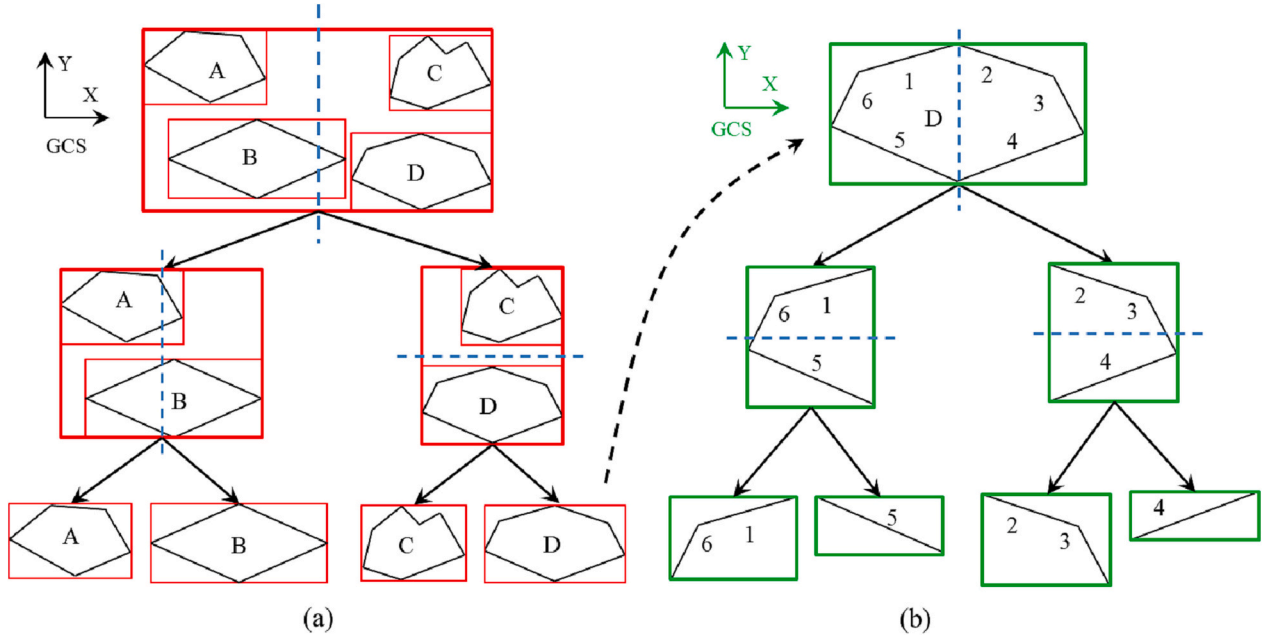


Fig. 10. 2D illustration of two-tier AABB tree indexing strategy: (a) indexing of a set of objects; (b) indexing of the triangle mesh of an individual object. The image shows the middle point-based tree node splitting and is adapted from [67].

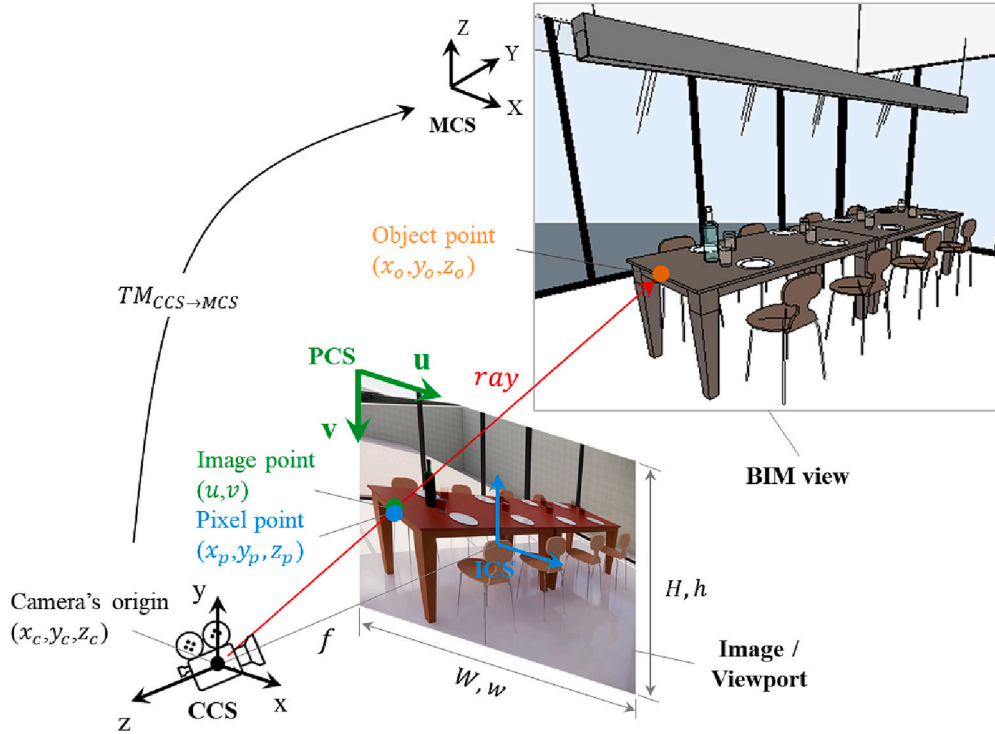


Fig. 11. Ray tracing to detect the corresponding object point of a pixel. The coordinates of the camera's origin, pixel point, and object point are all in the MCS.

coordinates of the pixel point in the MCS. Thus, the direction vector (r_d) of the ray can be obtained by:

$$r_d = \frac{1}{d} \cdot \begin{pmatrix} x_p - x_c \\ y_p - y_c \\ z_p - z_c \end{pmatrix} \quad (12)$$

where $d = \sqrt{(x_p - x_c)^2 + (y_p - y_c)^2 + (z_p - z_c)^2}$.

Once the ray is constructed, it is traced by traversing the two-tier

AABB tree of all the BIM objects in the VF to detect the closest intersection point with an object face. Fig. 12 summarizes the main steps of the ray tracing process.

First, candidate objects that the ray may intersect are searched by traversing the model AABB tree (Line 4 in Fig. 12). The traversal starts from the root node of the tree by checking whether the ray intersects the root AABB by using the ray-AABB intersection test [74]. If it does not, an empty set is returned. Otherwise, the traversal proceeds to the child nodes and repeats that test for each node recursively. The traversal

Algorithm 1: Ray tracing to annotate an image pixel

Input: Model-level AABB tree: $mtree$, where each object o carries an object-level AABB tree $otree$; Ray: r .

Output: Object class: cls
Object GUID: id
Object part class: cls_p
Object part ID: id_p
Object material: mal
Depth: d
Object point: op

- 1: Initialize intersected point set $pts = \{\}$
- 2: Initialize intersected triangle set $tris = \{\}$
- 3: Initialize distances $dist = \{\}$
- 4: Candidate objects $cos \leftarrow \text{TraverseModelTree}(r, mtree)$
- 5: **foreach** o **in** cos **do**
- 6: Candidate triangles $cts \leftarrow \text{TraverseObjectTree}(r, o.otree)$
- 7: Hit triangle, point and distance $t, p, dist \leftarrow \text{ComputeFirstHit}(r, cts)$
- 8: $tris.Add(t), pts.Add(p), dists.Add(dist)$
- 9: **end foreach**
- 10: First-hit object, triangle, and point $fo, ft, fp \leftarrow \text{FindFirstHit}(dists, tris, pts)$
- 11: $cls, id, cls_p, id_p, mal, d, op \leftarrow \text{ExtractAnnotations}(fo, ft, fp)$
- 12: **return** $cls, id, cls_p, id_p, mal, d, op$

Fig. 12. Ray tracing algorithm to annotate an image pixel.

progresses along the branches where nodes pass the test until leaf nodes are reached. All the BIM objects attached to the reached leaf nodes are gathered as candidate objects of the ray.

Next, the nearest intersection between the ray and each candidate object is computed separately (Lines 5–9 in Fig. 12). Specifically, this step first traverses each object's AABB tree to obtain candidate triangles that the ray may intersect. The traversing of an object tree is identical to that of the model tree. The range of candidate triangles are further narrowed by removing triangles whose back side faces towards the origin of the ray. Such a triangle can be identified simply if $r_d \cdot n > 0$, where n refers to the normal vector of the triangle. Then, for each candidate triangle, a fast and robust ray-triangle intersection algorithm

[75] is applied to compute the intersection. After all the intersections are obtained, the closest intersection with a specific object can be acquired by computing the Euclidean distance with the endpoint of the ray.

Lastly, the nearest intersections of all candidate objects are gathered and compared to find the closest intersection, i.e., the first-fit object point. Other associated information including object and/or object part class and instance ID and surface material can be obtained.

After repeating the ray tracing process described above for all the pixels of the synthetic image, the acquired pixelwise information is sufficient to generate the following annotations:

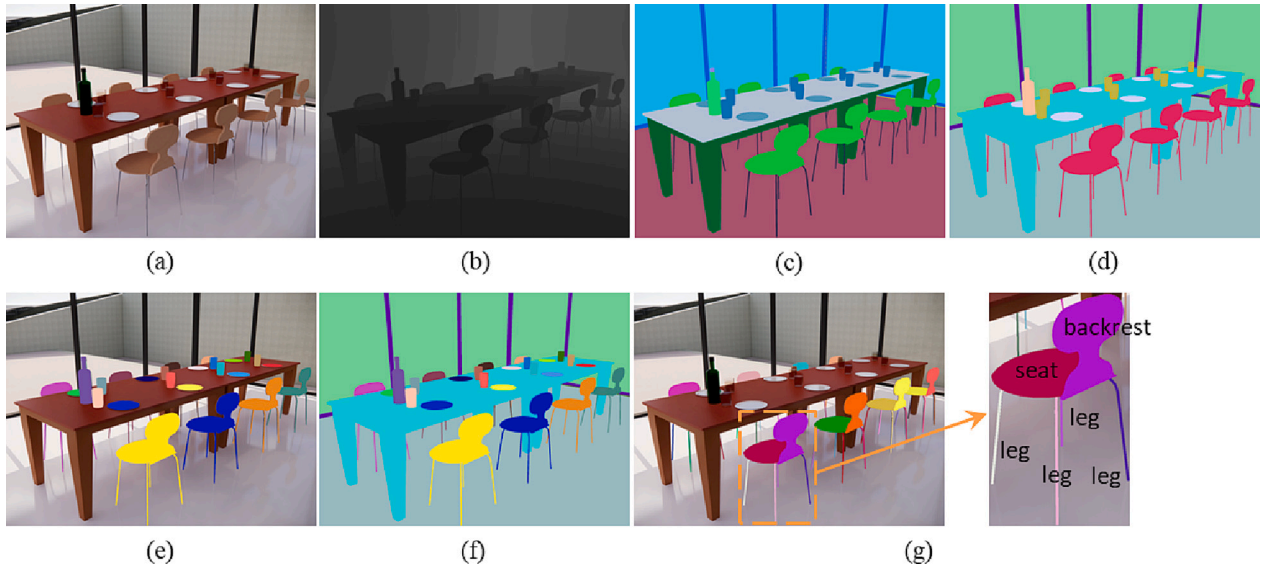


Fig. 13. BIM rendering image (a), depth annotation (b), and annotations for material semantic segmentation (c), object semantic segmentation (d), object instance segmentation (e), panoptic segmentation (f), and object part instance segmentation (g). Note: different object types/instances are colored differently; in (e) and (f), chairs, cups, plates, and bottles are set as the objects of interest for instance segmentation; in (g), for clarity, parts of chairs are displayed.

- Depth: the depth data of the entire BIM image is efficiently stored in a same-sized 16-bit grayscale image (see Fig. 13(b)), in which 3 times the depth value in millimeters is encoded as the brightness value. The largest depth that can be stored is 21.845 m, which is sufficient for building indoor scenes.
- Material semantic mask: pixels are annotated with the material type, and pixels with the same material type are grouped to create material semantic masks (see Fig. 13(c)).
- Object semantic mask: pixels are annotated with the object class, and pixels with the same object class are grouped to create object semantic masks (see Fig. 13(d)).
- Object instance mask: pixels are annotated with the object instance tag consisting of object class and instance ID, and pixels that share the same object instance tag are grouped to create object instance masks (see Fig. 13(e)).
- Object part instance mask: pixels are annotated with the object part instance tag, which consists of object part class and instance ID, and pixels that share the same object part instance tag are grouped to create object part instance masks (see Fig. 13(g)).
- Annotation for panoptic segmentation: as explained in Section 2.1, panoptic segmentation segments specific objects on the instance level and all other objects in an image on the semantic level. Therefore, annotations for panoptic segmentation are prepared by making (1) instance masks for those specified object types, and (2) semantic masks for all other objects in the image (see Fig. 13(f)).

The 3D object points of all the pixels constitute a point cloud corresponding to the synthetic image. Similarly, the point cloud can carry multiple annotations for scene recognition, semantic segmentation, instance segmentation, and part segmentation in 3D, as shown in Fig. 14. Furthermore, as described below, the annotated point cloud is used to generate the annotations for 3D object and object part detection.

3.5. Annotation of bounding box information

The remaining annotations needed include 2D BBs for 2D object and object part detection and 3D OBBs for 3D object and object part detection.

3.5.1. 2D BBs for 2D object and object part detection

2D object and object part detection tasks require that objects and object parts of interest in a training image are annotated with a 2D BB. Fig. 15 shows the 2D BB annotations of a generated synthetic image. A 2D BB of an object (or object part) refers to the smallest axis-aligned box that bounds the object (or object part), which can be completely defined by two vertices of the box in a diagonal in the image's PCS (see Fig. 15(a)). Given the known instance mask of an object (or object part) $\{(i, j)\}$, where (i, j) refers to a pixel's coordinates, the corresponding 2D BB $(u_{min}, v_{min}, u_{max}, v_{max})$ can be obtained by

$$\begin{cases} u_{min} = \min(\{i\}) \\ v_{min} = \min(\{j\}) \\ u_{max} = \max(\{i\}) \\ v_{max} = \max(\{j\}) \end{cases} \quad (13)$$

3.5.2. 3D OBBs for 3D object and object part detection

As discussed earlier, in 3D data such as point clouds and RGB-D images, the object and object part detection tasks aim to estimate a 3D OBB for each object and object part of interest, respectively. This study computes the 3D OBB of an object (or object part) based on its point cloud cluster generated in Section 3.4.3 by using the Open3D library [76]. Fig. 16(a) and (c) show the generated OBBs of objects and object parts in the point clouds, respectively. Note that we do not calculate 3D OBBs from the complete BIM geometries because 3D object detection aims to detect OBBs based on objects' or object parts' visible sections in the point clouds or RGB-D images.

Estimating OBBs of objects or object parts from a single monocular image has received increasing attention. To support this line of development, this study also generates OBB annotations for objects and object parts in the generated 2D synthetic images, as shown in Figs. 16(b) and (d). The method is to project the 3D OBBs obtained from a point cloud onto the corresponding synthetic image based on the perspective projection geometry. A 3D OBB is defined by six vertices in order in the MCS. Therefore, the corresponding OBB in the image is computed by projecting those vertices on the image. For a vertex (v_x, v_y, v_z) of a 3D OBB, the corresponding pixel vertex (u, v) on the image is calculated by

$$\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f \frac{W}{w} & 0 & \frac{W}{2} & 0 \\ 0 & -f \frac{H}{h} & \frac{H}{2} & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \cdot TM_{CCS \rightarrow MCS}^{-1} \cdot \begin{bmatrix} v_x \\ v_y \\ v_z \\ 1 \end{bmatrix} \quad (14)$$

where the first matrix on the right-hand side refers to the intrinsic matrix of the virtual camera, constructed by parameters including focus (f), width (W) and height (H) of the image in pixel, and width (w) and height (h) of the camera's viewport; $TM_{CCS \rightarrow MCS}$ refers to the known transformation matrix from the CCS to the MCS.

4. Implementation and validation

4.1. Software development

A Revit plug-in called BimImageNet has been implemented in C# and Python using the Revit API to demonstrate the proposed approach. It consists of two modules: image capture and image labeling. Their interfaces are shown in Fig. 17.

The image capture module encompasses functions to compile BIM perspective views using grids and to generate valid and distorted photorealistic images from the views. The former function is implemented in

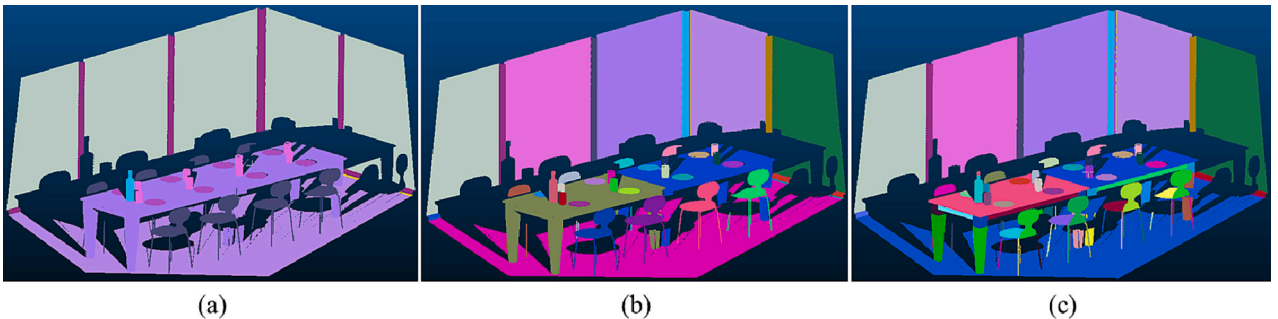


Fig. 14. Annotated point cloud for semantic segmentation (a), instance segmentation (b), and object part instance segmentation (c). Different object types/instances are colored differently.

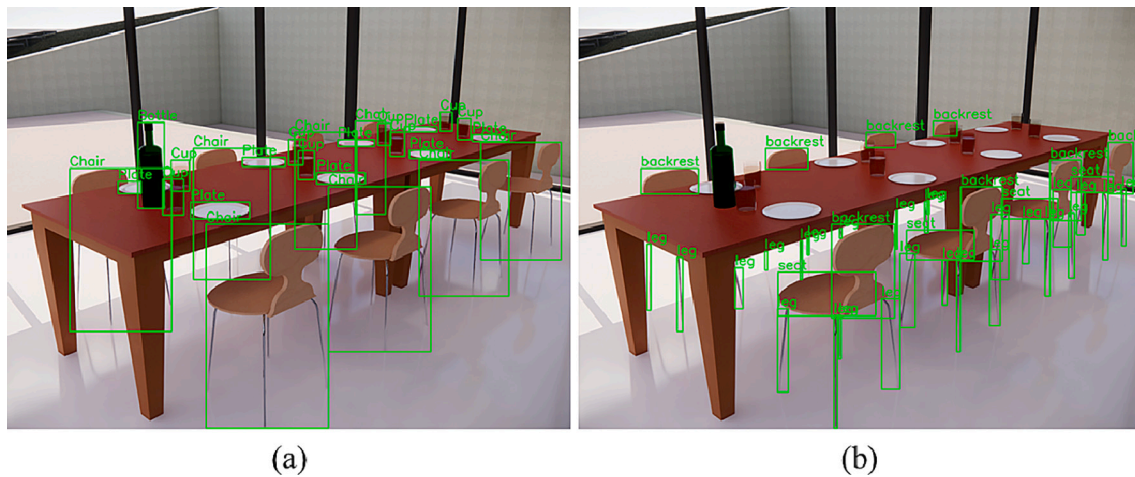


Fig. 15. 2D BBs for object (a) and object part (b) detection. For clarity, in (a), only BBs of chairs, cups, plates, and bottles are shown; in (b), only parts of chairs are shown.

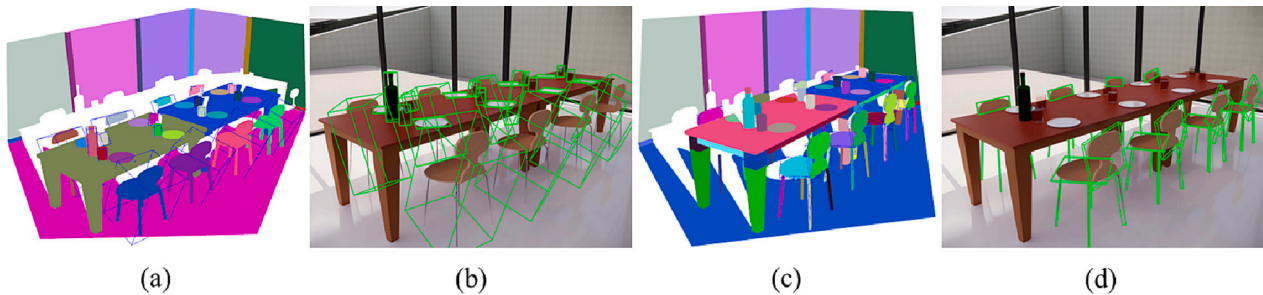


Fig. 16. 3D OBBs of objects in (a) point cloud and (b) image and 3D OBBs of object parts in (c) point cloud and (d) image. For clarity, in (a) and (b), only OBBs of desks, chairs, cups, and plates are shown; in (c) and (d), only OBBs of parts of chairs are shown.

C# and runs in the Revit run-time environment, from which the virtual camera of Revit is accessed, and all the needed BIM data is retrieved to generate perspective views for each selected space. After rendering the captured views into photorealistic images using Enscape, which works as a Revit plug-in, the latter function identifies valid images and distorts them to produce the desired synthetic images. This function involves extensive image processing operations and is implemented as an independent Python application that can be called by the image capturing module automatically. As shown in Fig. 17(b), this module allows users to customize a series of parameters required by both functions that can simulate different cameras to produce various images. It also enables users to select target spaces conveniently, based on the established linkage between space selection in the module interface and automatic localization and display of selected spaces in the Revit graphic user interface.

The image labeling module generates all the annotations required by the twelve BSU tasks for the obtained valid and distorted synthetic images. This module includes the pixelwise annotation function that uses ray tracing and the function for generating 2D/3D object and object part BBs. The former function is implemented in C# and the latter is a Python program that can be called by the image labeling module. As seen in Fig. 17(c), this module requires users to specify the object types for semantic- and instance-level annotations to prepare annotations for panoptic segmentation.

To improve the efficiency of the entire workflow, in both modules, all the functions that can be parallelized (capturing model views, identifying BIM objects in VF, object-level AABB tree creation, and ray tracing) are implemented in the multithreading mode to make the best use of a CPU's computing power. Although the approach is implemented as a Revit plug-in, it can be migrated easily to other BIM software or 3D

modeling tools.

4.2. Evaluation of the approach

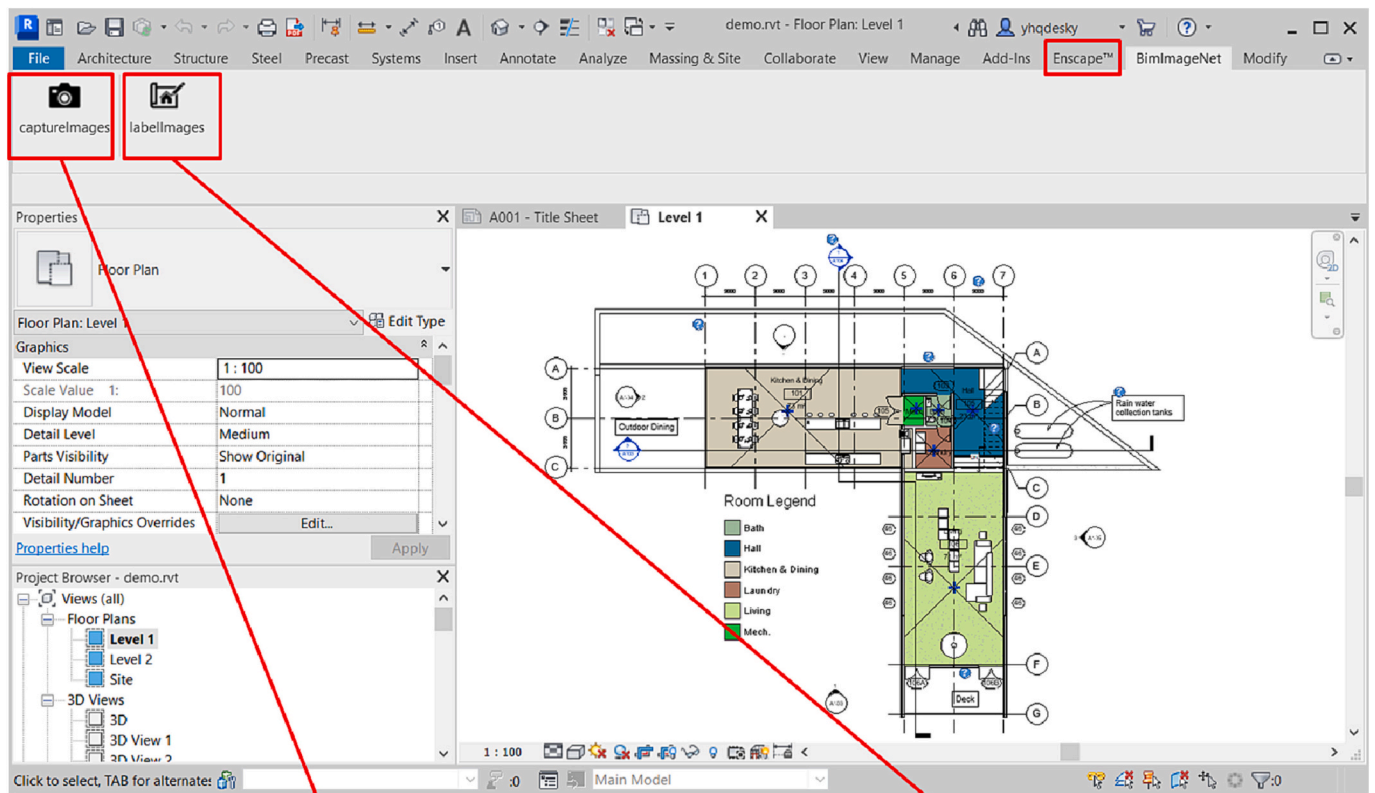
4.2.1. Test model preparation

A series of experiments were performed to evaluate the proposed approach. All the experiments were run on a desktop equipped with an Intel Core i7-8700K CPU and a RAM of 48 GB. Two BIM models (shown in Fig. 18) were used in the experiments: (1) a house building (Model A) that contains structural objects as well as common furniture and home appliances; and (2) large-scale equipment rooms of a commercial building (Model B) that contain various MEP objects such as pipes, ducts, lighting fixtures, sprinklers, and mechanical equipment. Eleven distinct spaces with eight types (i.e., living room, kitchen, bathroom, bedroom, laundry room, hall, dining room, and mechanical room) and various degrees of scene complexity were selected as target spaces (ten from Model A and one from Model B). The parameters required were pre-defined, as summarized in Table 1.

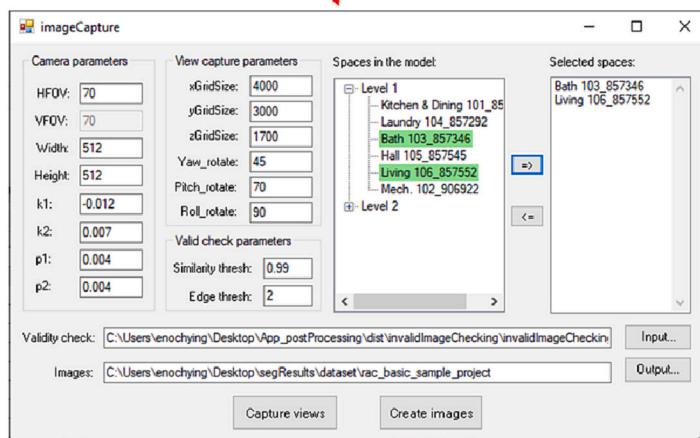
4.2.2. Accuracy evaluation

For all the selected spaces from the two models, our approach successfully sampled 108 feasible locations for placing the virtual camera and captured a total of 2208 RGB images, of which 706 were invalid. All 1502 valid images were labeled with the desired annotations required. Specifically, 245,727 object-part level instances were generated, covering >40 object types and >42 material types. To better demonstrate the results, two additional views that depict scenes with varying levels of complexity were captured, rendered, and annotated into two larger-sized (1024 × 768) images, as shown in Figs. 19 and 20.

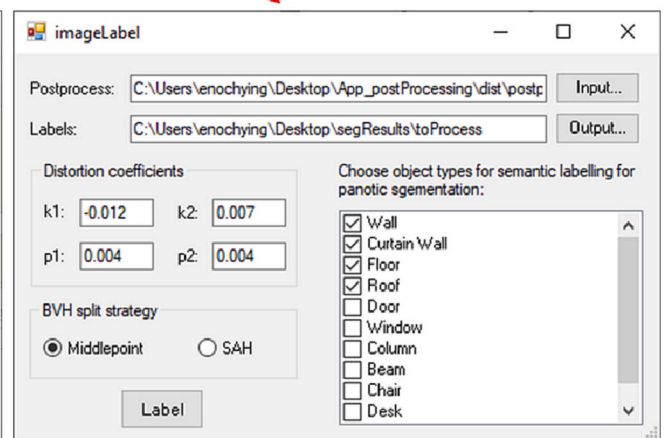
The annotations of all the 1502 valid images were checked manually



(a)

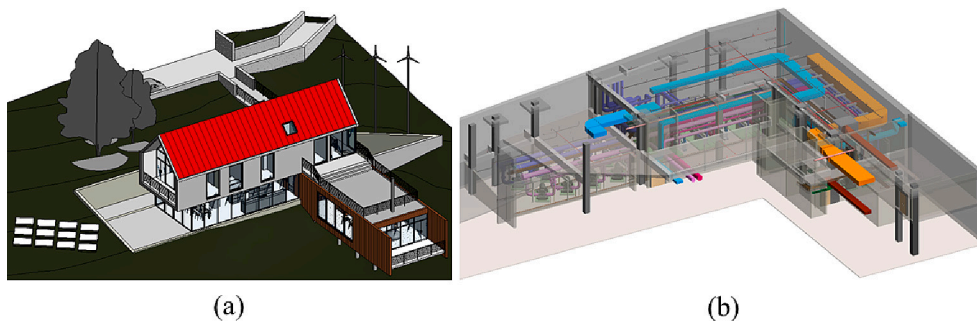


(b)



(c)

Fig. 17. The developed Revit plug-in (a), image capturing module interface (b), and image labeling module interface (c).



(a)

(b)

Fig. 18. Two test BIM models: (a) Model A; (b) Model B.

Table 1
Configuration of relevant parameters.

Parameters	Model A	Model B
FOVs in degrees (HFOV, VFOV)	(70, 70)	(70, 70)
Image size in pixels (Width, Height)	(512, 512)	(512, 512)
Distortion coefficients (k1, k2, p1, p2)	(−0.012, 0.007, 0.004, 0.004)	(−0.012, 0.007, 0.004, 0.004)
Grid sizes in mm (xGridSize, yGridSize, zGridSize)	(1000/2000/2500/3000/ 4000/5000, 1500/3000/5000, 1700/2000)	(6000, 6000, 1700)
Rotation angles in degrees (Yaw, Pitch, Roll)	(45, 70/90, 90)	(45, 70, 90)
Similarity threshold	0.99	0.99
Edge threshold	2	2
Tree node split strategy	Middle point	Middle point
Semantic labels for panoptic segmentation	{Wall, Curtain Wall, Slab, Roof}	{Wall, Slab, Roof}

Note. (1) The camera-related parameters including FOVs, image size, and distortion coefficients were assigned arbitrarily for demonstration. (2) The view capture parameters, including grid sizes and rotation angles, were determined based on the dimensions of the target spaces and the complexity of the model scenes: big grid sizes were set for large spaces and small rotation angles for complex model scenes. The parameter value a/b/c means that relevant parameters were set with a, b, and c for different spaces. (3) A high similarity threshold and a small edge threshold were set to eliminate the misidentification of valuable images by the valid image checking function.

and visually. For scene type and camera pose, we checked the existence of the annotations. Scene types are directly extracted from BIM models and camera poses are set by the approach in a straightforward manner: hence, they are deemed accurate if they exist. For the annotation of pixel depth, since calculating the ground truths manually is impracticable, we

visually checked the generated depth image (see the example of Fig. 19 (c)) by comparing it with the corresponding rendered image. The annotations of the remaining BSU tasks - object and material semantic segmentation, 2D and 3D object and object part detection, object and object part instance segmentation, and panoptic segmentation - consist of two parts: boxes or masks that localize objects in the image and corresponding class or instance labels. For convenience, we drew boxes (see Figs. 19(d)-(h)) and overlaid masks in various colors, each representing different object/material classes or instances (see Figs. 19(i)-(l)), on the rendered images to facilitate visual checking. The class and instance labels were manually checked at the same time.

The checking results show that all the generated annotations are accurate, with just two typical exceptions:

- (1) A few erroneous mask annotations (for semantic segmentation and instance segmentation) were caused by a geometry clash where a floor had a coplanar surface with a ceiling in Model A. The approach annotated the pixels of the common surface as a mix of “Floor” and “Ceiling” because sometimes the floor was detected as the first-hit object by the ray tracing algorithm, and at other times the ceiling object was detected first.
- (2) A considerable number of object part class labels (for 2D and 3D object part detection and object part instance segmentation) were missing because the data was absent in the BIM models. For example, in Fig. 19(g), although the approach accurately generated the masks of all the parts of the chairs in the image, the class labels of the part instances could not be determined because they are missing from the BIM model.

These annotation errors are all caused by quality issues in the input BIM models rather than by limitations in our approach. Thus, in general, the experiments demonstrate the accuracy and robustness of our

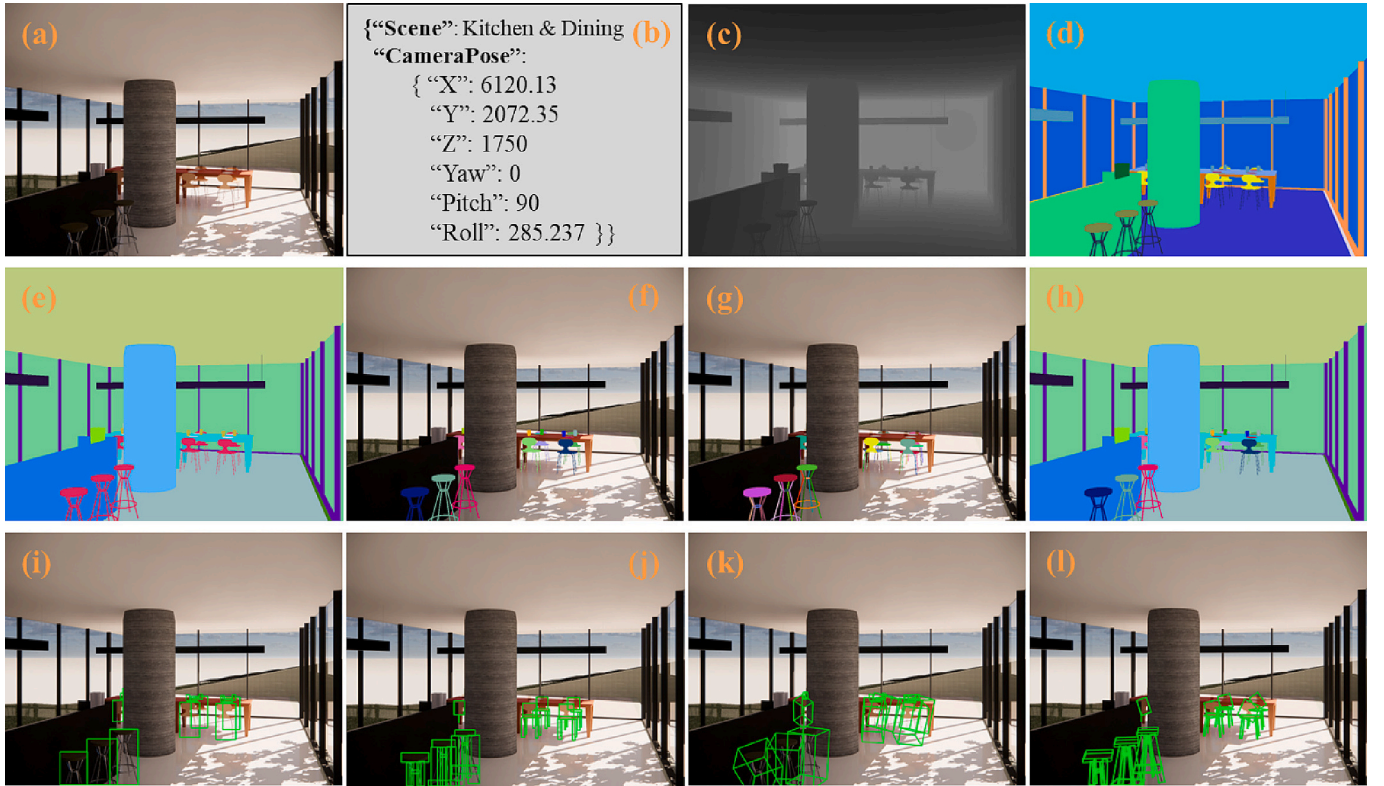


Fig. 19. Rendering image generated by Model A (a) and its annotations: (b) scene and camera pose; (c) depth; (d) material mask; (e) object semantic mask; (f)(g) object and object part instance; (h) annotation for panoptic segmentation; (i)(j) 2D BBs for object and object part detection; and (k)(l) 3D BBs for object and object part detection. For clarity, in (f), (g), and (i), only the annotations of small objects including chairs, plates, cups, and bottles are shown; in (j), (k), and (l), only the annotations of parts of chairs are shown.

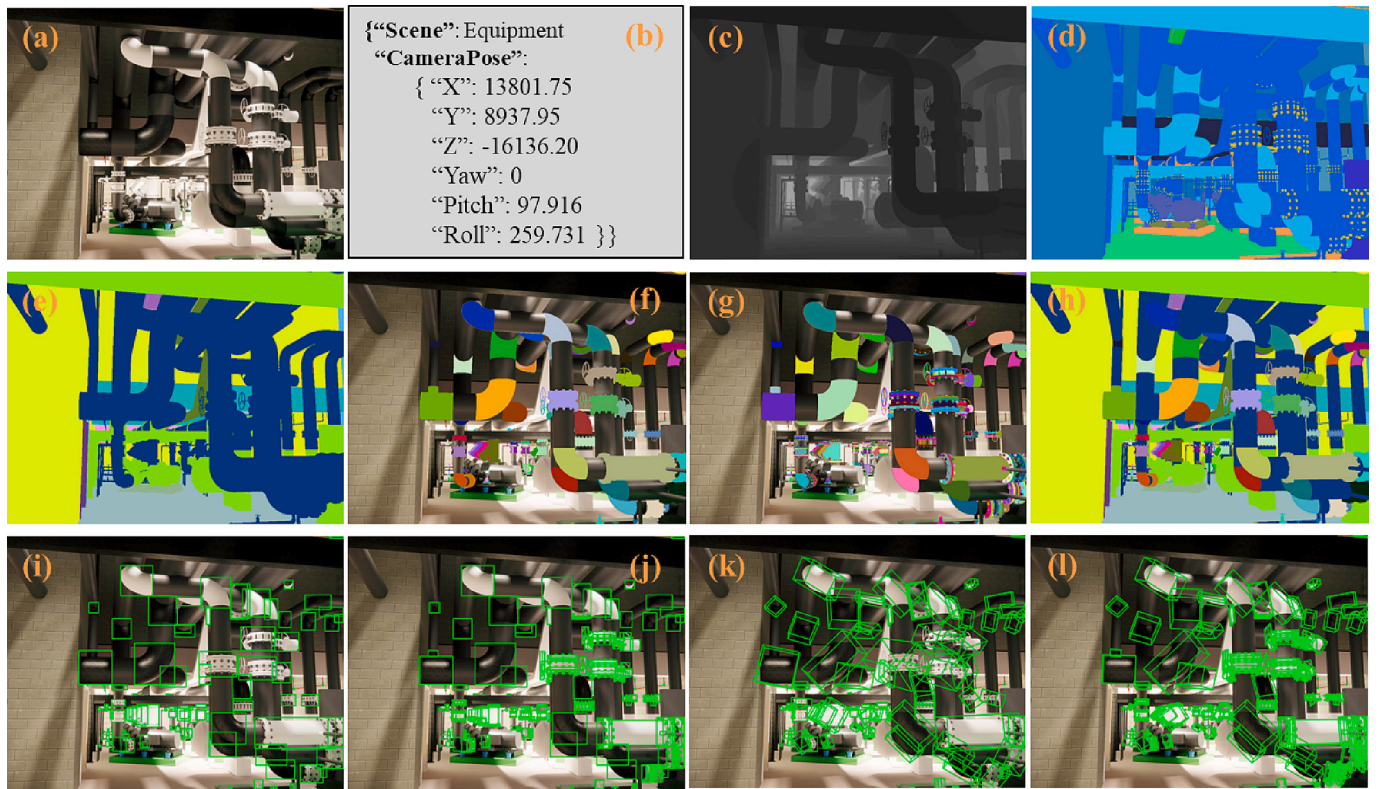


Fig. 20. Rendering image generated from Model B (a) and its annotations: (b) scene and camera pose; (c) depth; (d) material mask; (e) object semantic mask; (f)(g) object and object part instance; (h) annotation for panoptic segmentation; (i)(j) 2D BBs for object and object part detection; and (k)(l) 3D BBs for object and object part detection. For clarity, in (f), (g), (i), (j), (k), and (l), only the annotations of pipe fittings, pipe accessories and/or their parts are displayed.

approach in terms of generating annotated synthetic images of indoor BIM scenes.

4.2.3. Efficiency evaluation

The total running time (including view capture, image rendering, image distortion, image validity checking, image annotation, and annotation output) for generating and annotating the 1502 valid images with 245,727 object-part level instances from the two BIM models was about 12 h, representing an average of 28 s for one valid image with 164 object part instances. This is far more efficient than annotating a similar real image manually.

To further evaluate the efficiency of the approach, additional experiments on single image generation and annotation were performed. The run time needed to create an annotated image depends primarily on two factors: the size of the output image and the size of the scene in the VF of the virtual camera. Larger output images mean more pixels to be distorted and annotated. Larger scenes contain more objects in the VF that need to be examined for annotation computation. Hence, different settings were used for the image size and the scene size in the experi-

ments. These can be quantified by the number of pixels in the image and the number of triangles in the scene, respectively. Specifically, two scenes on different scales were used, as shown in Fig. 19 (Scene A with 55,328 triangles) and Fig. 20 (Scene B with 1,962,784 triangles). For each scene, six images with different sizes (256×192 , 512×384 , 768×576 , 1024×768 , 1280×960 , and 1536×1152) were generated and annotated using the software implementation. Table 2 shows the run times, which cover the steps of image rendering (IR), image distortion (ID), ray tracing-based pixelwise annotation (RT), and other annotations (OA). The view capture and image validity checking steps were not relevant in this experimental scenario.

As shown in Table 2, rendering an image using Enscape is fast and takes an almost constant time (i.e., 3 s; see the IR column), regardless of image size and scene complexity. In fact, Enscape renders a model view almost in real-time and the IR value mainly refers to the time required to output a rendered image. The durations for distorting images (ID) and for generating object and object part BBs (OA) increase linearly with the image size, and are negligible compared to the time required for ray tracing for pixelwise annotation (RT). This is the most time-consuming

Table 2

Run times (in seconds) for processing the selected scenes with different image sizes.

Image size	Scene A (55,328 triangles) (base)					Scene B (1,962,784 triangles) ($\times 35$)				
	IR	ID	RT	OA	Total	IR	ID	RT	OA	Total
256×192 (base)	3	0.2	7.5	0.4	11.1	3	0.3	46	1	50.3
512×384 ($\times 4$)	3	0.9	20.2	1.7	25.8	3	1.3	77	5	86.3
768×576 ($\times 9$)	3	2.2	41.8	3.9	50.9	3	3	133	12	151
1024×768 ($\times 16$)	3	4	71.4	6.5	84.9	3	5.3	217	44	269.3
1280×960 ($\times 25$)	3	6	108.6	10.3	127.9	3	8.2	329	39	379.2
1536×1152 ($\times 36$)	3	8.7	155	15	181.7	3	11.8	489	59	562.8

Note. The image size is represented as width $\times$ height in pixels; the symbol “($\times$ number)” means the scene or the image is “number” times larger than the base scene or the base image.

step and consumes up to 87% of the total run time (in the case of the image size of 1536×1152 for *Scene B*).

For the same scene, the total run time generally increases with the image size, but not linearly. For example, the largest image of *Scene A* is 36 times larger than the smallest base image, while the running time increases only by a factor of about 16. The same phenomenon can be observed in the scene size. For instance, *Scene B* is 35 times larger than *Scene A*, while among all the image sizes, the running times increase by a factor of 5 at most (in the case of the image size of 256×192). This can be explained by the acceleration effect introduced by the two-tier AABB tree indexing in the most time-consuming ray tracing step. It appears therefore that the two-tier AABB tree indexing tactic is essential for our approach to handle complex BIM model scenes and to cope with large-sized images.

Nevertheless, for the purpose of large-scale dataset generation, further efficiency enhancement would be desirable. As seen in the table, the prototype takes 51 s to generate and label a modest-size (768×576) image that describes a typical model scene (i.e., *Scene A* with 55,328 triangles).

4.3. Preliminary validation of effectiveness

Generating synthetic data has value only if the images can be used effectively for BSU tasks. We therefore carried out a set of experiments as a preliminary check of the effectiveness of the synthetic data. We prepared real and synthetic datasets and compared their performance in object instance segmentation with a deep learning model (Mask R-CNN [53], a typical and widely used image segmentation algorithm).

We constructed a real training dataset with 400 real images, a synthetic training dataset with 400 synthetic images, and a real test dataset with 80 real images. The real training images were captured from the interiors of four university office buildings and the real test images from the interiors of 10 other university office buildings. The synthetic training images were automatically generated by our approach from 10 BIM models of other residential and office buildings. In all these datasets, four common interior building elements (wall, door, floor, and ceiling) were labeled. Labelling annotations was done manually in real training and real test datasets, and automatically using our approach in the synthetic training dataset.

Furthermore, to explore the effectiveness of augmenting synthetic data with a small amount of real data, we prepared two additional training datasets:

- A mixed training dataset with 440 images that contains all the synthetic training images and 10% of the real training images, randomly selected from the 400 real images.
- A mixed training dataset with 480 images that contains all the synthetic training images and 20% of the real training images, randomly selected from the 400 real images.

In each training dataset, 90% of the images were used for training, and the remaining 10% were used for validation. The Mask R-CNN model pretrained on the large-scale COCO image dataset [88] was used as the same starting point for each training dataset. To ensure fairness, all the training was implemented on the same computing device (i.e., a desktop with 64GB RAM and a NVIDIA RTX 3060 GPU with 12 GB memory) and used the same training strategy (i.e., two-stage early-stopping training strategy [54]) and hyperparameters (e.g., batch size of 2, learning momentum of 0.9, weight decay of 0.0001, learning rate (LR) of 0.001 for 60 epochs in the first stage, LR of 0.0001 and 0.00002 for the first 90 epochs and the remaining epochs in the second stage).

Table 3 shows the results of each trained model on the same test dataset. The fully synthetic dataset has weaker performance than the same dataset supplemented with 10% or 20% of real images, both of which have very similar performance to that of the real image dataset, for both precision and recall. This suggests that a method that replaces

up to 90% of the real images with synthetic images is potentially highly effective, as well as being highly efficient.

Note also that the actual performance gap may be smaller because the reality gap between the real training dataset and the real test dataset is smaller than the gap between the synthetic training dataset and the real test dataset. This is because the real training images and most of the real test images were captured from office buildings in the same university, while synthetic images were generated from BIM models of other residential and office buildings.

5. Discussion

5.1. Prerequisites of the approach

The proposed approach enables the compilation of large numbers of fully labeled photorealistic images in a short time by rendering BIM model scenes. Naturally, the quality of the chosen input BIM models has a significant impact on the realism of the resulting images and the accuracy of their annotations.

The realism of the resulting images is directly dependent on that of the BIM model scenes, which includes two aspects. The first relates to the realism of the geometry and appearance (i.e., material and textures) of individual BIM objects. An ideal BIM scene should reflect as-constructed or as-built building scenes with high level-of-detail geometry and realistic material and textures for objects included. Furthermore, the illumination-related objects and data such as building geolocation and lighting fixtures and relevant properties (e.g., location and shape of light source, brightness, intensity, color, and light distribution mode) should be properly defined in the BIM models. All these enable the rendering engine to generate *appearance-realistic* images. The second aspect relates to the realism of the holistic scenes. Building scenes represented by BIM models are often cleaner or more well-organized than real-world scenes, where random objects can be found and the locations of moveable objects like desks and chairs can be arbitrary. It is thus necessary to introduce some “mess” to BIM models, e.g., adding random objects and placing movable objects randomly, to make the captured images more *scene-realistic*.

As revealed by the experiments, the accuracy of image annotations can be affected by quality issues of BIM models. One common issue is geometric clashes between BIM objects, which do not occur in real-world building scenes. Deep learning models trained on synthetic images with scenes that cannot exist in reality may lead to unexpected errors in understanding real-world images. Furthermore, geometric clashes can also lead to erroneous annotations (see Section 4.2.2). Therefore, geometric clashes in BIM models should be identified and corrected before applying this approach. Nowadays, many software packages like Solibri [77] offer effective functions to detect geometric clashes in BIM models.

Another major quality issue occurs when object and object part class names are incorrect, missing, inconsistent or too generic. For example, desks and chairs in Revit are generally classified as “Furniture” without further distinction; different parts of a chair in Revit are not given specific class names; and the wall's class name in Revit is “Walls” but in ArchiCAD it is “Wall”. To address the issues of inconsistent and generic class names, this study defined a small ontology that encompasses

Table 3

Performance of the Mask R-CNN model trained on different datasets.

Training dataset	AP (%)	AR(%)
Real	52.00	57.30
Synthetic	44.55	50.83
Synthetic +10% real	50.88	55.35
Synthetic +20% real	54.85	58.20

Note. AP and AR refer to mean average precision and mean average recall [88] over all the target object instances, respectively.

common architectural, furniture and MEP objects and applied a simple keyword-based method to map Revit objects and/or their parts to the ontology based on their Revit class type and the value of their Name property. Missing object or object part class information was assigned manually. In the future, a more comprehensive ontology of building objects will be developed by incorporating the generic lexical database WordNet [78] and existing building ontologies (e.g., IfcOWL [79] and Brick [80]) and classification systems (e.g., OmniClass [81] and Uniclass [82]), to support the creation of large-scale image datasets. Meanwhile, both rule-based [83] and machine learning [84,85] algorithms will be exploited to establish more generalized and dependable BIM object and object part classification methods.

5.2. Limitations and future work

Although the experiments demonstrate the accuracy and robustness of the proposed approach in terms of automatic synthetic image generation and annotation and the effectiveness of the generated synthetic images for the instance segmentation task, the approach and the study have some limitations that are worthy of further development.

In its current implementation, the image capturing module requires users to manually select distinct interior spaces of a BIM model for processing, to avoid capturing images of duplicate or similar indoor scenes. This would be rather laborious and time-consuming for large-scale building models that contain thousands of spaces. An efficient automated method that can check space similarity and filter desired spaces is needed. Furthermore, this module requires users to input a series of parameters, including horizontal and vertical grid intervals and rotation angles (i.e., yaw, pitch, and roll), to generate feasible poses of the virtual camera in each selected space. Setting smaller values for these parameters means more images can be captured. However, it also means that more time is needed for valid image checking and labeling. Hence, future research may attempt to develop appropriate optimization algorithms for these parameters.

The image labeling module also has room for improvement in efficiency. As shown in Section 4.2.3, the embedded ray tracing process could consume over 80% of the total run time of the proposed approach. Given the parallel computing nature of ray tracing, GPU-based implementation would significantly improve the efficiency of the approach, thereby benefiting the creation of large-scale datasets containing hundreds of thousands of images.

Lastly, the effectiveness of the synthetic images generated by the approach needs to be more systematically and rigidly examined. Currently, we have only assessed the effectiveness of the synthetic images for the instance segmentation task through small-scale experiments, considering only four building objects and one deep learning model. In the future, both synthetic and real-world image datasets with a considerable size covering more object types should be created and tested with more commonly-used deep learning models. Likewise, the effectiveness of the synthetic images for the other eleven BSU tasks should be examined in a similar manner.

6. Conclusions

Deep learning-based building scene understanding (BSU) algorithms often require large-scale well-annotated datasets for training. Creating such datasets involves tedious real-world data collection and costly and laborious data labeling. This study proposes an automatic BIM-based synthetic image generation approach that can capture photorealistic images of BIM model scenes and compute various types of annotations for twelve BSU tasks, including camera pose estimation, scene recognition, depth estimation, 2D/3D object and object part detection, object and material semantic segmentation, object and object part instance segmentation, and panoptic segmentation. The approach includes a parameterized method to determine virtual camera poses to capture 3D perspective views of indoor BIM model scenes. The captured views are

further rendered with random date and time, checked, and distorted to generate valid photorealistic images with diverse visual effects. The annotations of these images required by the above BSU tasks are computed by a ray tracing-based algorithm with relevant postprocessing steps. A two-tier AABB tree indexing strategy is adopted to expedite the expensive ray tracing process.

Our approach was implemented as a Revit Plug-in, which was used to process 11 distinct and complex indoor BIM model scenes. A total of 1502 valid synthetic images containing 245,727 object part instances were generated automatically in about 12 h. These images were annotated accurately except for some errors caused by the quality issues of the input BIM models. This demonstrates the accuracy, robustness, and efficiency of the approach. Furthermore, preliminary experiments show that compared to real images, the synthetic images generated by our approach demonstrate very promising effectiveness for the challenging BSU task of building object instance segmentation. Therefore, our approach can provide a useful and reliable tool for researchers and practitioners to prepare their own synthetic image datasets for specific BSU tasks in a cost-efficient way.

Potentially, the approach can have two useful extensions. It currently captures discrete RGB-D images instead of spatio-temporal continuous RGB-D frames produced by a real RGB-D camera, which have proven to be particularly useful for engineering applications such as 3D reconstruction and navigation. The first extension is thus to add a camera motion simulation module to upgrade the used virtual camera to be an RGB-D camera. The second extension is to adapt the current static BIM data extraction method to dynamic 4D BIM models. This would make it possible to generate synthetic image datasets for buildings under construction, which could greatly facilitate the use of deep learning techniques for construction scene understanding.

This study showcases an emerging application area of BIM, i.e., how the rich and well-structured information stored in BIM models can be extracted and transformed to generate large amounts of well-annotated training data automatically and cost-effectively. This data facilitates the use of AI, particularly deep learning techniques in the AECO industry. Although this study focused on creating synthetic image data for image-based BSU tasks, BIM data can also be converted into other formats such as point clouds and graphs for use in deep learning algorithms, and thus serving a broad range of engineering tasks such as scan-to-BIM and digital twin development.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The source code of Revit Plugin and all the experiment data will be published soon on the GitHub at <https://github.com/EnochYing/BimImageNet>

Acknowledgment

This research was supported by the Ministry of Science & Technology, Israel, under grant #3-15651.

References

- [1] B. Ekanayake, J.K.W. Wong, A.A.F. Fini, P. Smith, Computer vision-based interior construction progress monitoring: a literature review and future research directions, *Autom. Constr.* 127 (2021), 103705, <https://doi.org/10.1016/j.autcon.2021.103705>.
- [2] B. Sherafat, C.R. Ahn, R. Akhavan, A.H. Behzadan, M. Golparvar-Fard, H. Kim, Y. C. Lee, A. Rashidi, E.R. Azar, Automated methods for activity recognition of construction workers and equipment: state-of-the-art review, *J. Constr. Eng.*

- Manag. 146 (6) (2020) 03120002, <https://doi.org/10.1016/j.asceco.1943-7862.0001843>.
- [3] S. Tang, D. Roberts, M. Golparvar-Fard, Human-object interaction recognition for automatic construction site safety inspection, *Autom. Constr.* 120 (2020), 103356, <https://doi.org/10.1016/j.autcon.2020.103356>.
 - [4] A. Morar, A. Moldoveanu, I. Mocanu, F. Moldoveanu, I.E. Radoi, V. Asavei, A. Gradinaru, A. Buteanu, A comprehensive survey of indoor localization methods based on computer vision, *Sensors* 20 (9) (2020) 2641, <https://doi.org/10.3390/s20092641>.
 - [5] Q. Lu, S. Lee, L. Chen, Image-driven fuzzy-based system to construct as-is IFC BIM objects, *Autom. Constr.* 92 (2018) 68–87, <https://doi.org/10.1016/j.autcon.2018.03.034>.
 - [6] H. Fathi, F. Dai, M. Lourakis, Automated as-built 3D reconstruction of civil infrastructure using computer vision: achievements, opportunities, and challenges, *Adv. Eng. Inform.* 29 (2) (2015) 149–161, <https://doi.org/10.1016/j.aei.2015.01.012>.
 - [7] J. Deng, W. Dong, R. Socher, L.J. Li, K. Li, L. Fei-Fei, Imagenet: a large-scale hierarchical image database, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2009, pp. 248–255, <https://doi.org/10.1109/CVPR.2009.5206848>.
 - [8] T.Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, C.L. Zitnick, Microsoft coco: common objects in context, in: *European Conference on Computer Vision*, 2014, pp. 740–755, https://doi.org/10.1007/978-3-319-10602-1_48.
 - [9] B. Zhou, H. Zhao, X. Puig, S. Fidler, A. Barriuso, A. Torralba, Scene parsing through ade20k dataset, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017, pp. 633–641, <https://doi.org/10.1109/CVPR.2017.544>.
 - [10] B. Xiao, S.C. Kang, Development of an image data set of construction machines for deep learning object detection, *J. Comput. Civ. Eng.* 35 (2) (2021) 05020005, [https://doi.org/10.1061/\(ASCE\)CP.1943-5487.0000945](https://doi.org/10.1061/(ASCE)CP.1943-5487.0000945).
 - [11] X. An, Z. Li, Z. Liu, C. Wang, P. Li, Z. Li, Dataset and benchmark for detecting moving objects in construction sites, *Autom. Constr.* 122 (2021), 103482, <https://doi.org/10.1016/j.autcon.2020.103482>.
 - [12] H. Tajen, Z. Zhu, Image dataset development for measuring construction equipment recognition performance, *Autom. Constr.* 48 (2014) 1–10, <https://doi.org/10.1016/j.autcon.2014.07.006>.
 - [13] J. Hwang, J. Kim, S. Chi, J. Seo, Development of training image database using web crawling for vision-based site monitoring, *Autom. Constr.* 135 (2022), 104141, <https://doi.org/10.1016/j.autcon.2022.104141>.
 - [14] Amazon SageMaker Data Labeling Pricing, <https://aws.amazon.com/sagemaker/data-labeling/pricing/> (Last Accessed: February 10th, 2023).
 - [15] AI Platform Data Labeling Service, <https://cloud.google.com/ai-platform/data-labeling/docs> (Last Accessed: February 10th, 2023).
 - [16] C.G. Northcutt, A. Athalye, J. Mueller, Pervasive label errors in test sets destabilize machine learning benchmarks, *arXiv preprint (2021)*, <https://doi.org/10.48550/arXiv.2103.14749> (arXiv: 2103.14749).
 - [17] A. Handa, V. Patrauceanu, V. Badrinarayanan, S. Stent, R. Cipolla, Understanding real world indoor scenes with synthetic data, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 4077–4085, <https://doi.org/10.1109/CVPR.2016.442>.
 - [18] J. Papon, M. Schoeler, Semantic pose using deep networks trained on synthetic RGB-D, in: *Proceedings of the IEEE International Conference on Computer Vision*, 2015, pp. 774–782, <https://doi.org/10.1109/ICCV.2015.95>.
 - [19] Revit, <https://www.autodesk.com/products/revit/overview> (Last Accessed: February 10th, 2023).
 - [20] G. Ros, L. Sellart, J. Materzynska, D. Vazquez, A.M. Lopez, The synthia dataset: A large collection of synthetic images for semantic segmentation of urban scenes, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 3234–3243, <https://doi.org/10.1109/CVPR.2016.352>.
 - [21] I. Ha, H. Kim, S. Park, H. Kim, Image retrieval using BIM and features from pretrained VGG network for indoor localization, *Build. Environ.* 140 (2018) 23–31, <https://doi.org/10.1016/j.buildenv.2018.05.026>.
 - [22] D. Acharya, K. Kourosh, W. Stephan, BIM-PoseNet: indoor camera localization using a 3D indoor model and deep learning from synthetic images, *ISPRS J. Photogramm. Remote Sens.* 150 (2019) 245–258, <https://doi.org/10.1016/j.isprsjprs.2019.02.020>.
 - [23] D. Acharya, S. Singha Roy, K. Khoshelham, S. Winter, A recurrent deep network for estimating the pose of real indoor images from synthetic image sequences, *Sensors* 20 (19) (2020) 5492, <https://doi.org/10.3390/s20195492>.
 - [24] Y. Hong, S. Park, H. Kim, H. Kim, Synthetic data generation using building information models, *Autom. Constr.* 130 (2021), 103871, <https://doi.org/10.1016/j.autcon.2021.103871>.
 - [25] J.G. Lee, J. Hwang, S. Chi, J. Seo, Synthetic image dataset development for vision-based construction equipment detection, *J. Comput. Civ. Eng.* 36 (5) (2022) 04022020, [https://doi.org/10.1061/\(ASCE\)CP.1943-5487.0001035](https://doi.org/10.1061/(ASCE)CP.1943-5487.0001035).
 - [26] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, P. Abbeel, Domain randomization for transferring deep neural networks from simulation to the real world, in: *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2017, pp. 23–30, <https://doi.org/10.1109/IROS.2017.8202133>.
 - [27] J. Tremblay, A. Prakash, D. Acuna, M. Brophy, V. Jampani, C. Anil, T. To, E. Cameracci, S. Bochoon, S. Birchfield, Training deep networks with synthetic data: Bridging the reality gap by domain randomization, in: *Proceedings of Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 969–977, <https://doi.org/10.1109/CVPRW.2018.00143>.
 - [28] A. Assadzadeh, M. Arashpour, I. Brilakis, T. Ngo, E. Konstantinou, Vision-based excavator pose estimation using synthetically generated datasets with domain randomization, *Autom. Constr.* 134 (2022), 104089, <https://doi.org/10.1016/j.autcon.2021.104089>.
 - [29] K. Bousmalis, A. Irpan, P. Wohlhart, Y. Bai, M. Kelcey, M. Kalakrishnan, L. Downs, J. Ibarz, P. Pastor, K. Konolige, S. Levine, Using simulation and domain adaptation to improve efficiency of deep robotic grasping, in: *IEEE International Conference on Robotics and Automation*, 2018, pp. 4243–4250, <https://doi.org/10.1109/ICRA.2018.8460875>.
 - [30] M. Johnson-Roberson, C. Barto, R. Mehta, S.N. Sridhar, K. Rosaen, R. Vasudevan, Driving in the matrix: Can virtual worlds replace human-generated annotations for real world tasks?, in: *IEEE International Conference on Robotics and Automation*, 2017, pp. 1–8, <https://doi.org/10.1109/ICRA.2017.7989092>.
 - [31] Q. Li, R. Cao, J. Zhu, X. Hou, J. Liu, S. Jia, Q. Li, G. Qiu, Improving synthetic 3D model-aided indoor image localization via domain adaptation, *ISPRS J. Photogramm. Remote Sens.* 183 (2022) 66–78, <https://doi.org/10.1016/j.isprsjprs.2021.10.005>.
 - [32] J. Chen, S. Li, D. Liu, W. Lu, Indoor camera pose estimation via style-transfer 3D models, *Comput. -Aided Civ. Infrastruct. Eng.* 37 (3) (2022) 335–353, <https://doi.org/10.1111/mice.12714>.
 - [33] J.Y. Zhu, T. Park, P. Isola, A.A. Efros, Unpaired image-to-image translation using cycle-consistent adversarial networks, in: *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 2223–2232, <https://doi.org/10.1109/ICCV.2017.244>.
 - [34] Enscape, <https://enscape3d.com/> (Last Accessed: February 10th, 2023).
 - [35] v-Ray, <https://www.chaos.com/> (Last Accessed: February 10th, 2023).
 - [36] M. Naser, S. Khan, F. Porikli, Indoor scene understanding in 2.5/3d for autonomous agents: a survey, *IEEE Access* 7 (2018) 1859–1887, <https://doi.org/10.1109/ACCESS.2018.2886133>.
 - [37] A. Garcia-Garcia, S. Orts-Escolano, S. Oprea, V. Villena-Martinez, J. Garcia-Rodriguez, A review on deep learning techniques applied to semantic segmentation, *arXiv preprint (2017)*, <https://doi.org/10.48550/arXiv.1704.06857>.
 - [38] J. Crespo, J.C. Castillo, O.M. Mozos, R. Barber, Semantic information for robot navigation: a survey, *Appl. Sci.* 10 (2) (2020) 497, <https://doi.org/10.3390/app10020497>.
 - [39] G. Tian, L. Liu, J. Ri, Y. Liu, Y. Sun, ObjectFusion: An object detection and segmentation framework with RGB-D SLAM and convolutional neural networks, *Neurocomputing* 345 (2019) 3–14, <https://doi.org/10.1016/j.neucom.2019.01.088>.
 - [40] Z.Q. Zhao, P. Zheng, S.T. Xu, X. Wu, Object detection with deep learning: a review, *IEEE Transactions on Neural Networks and Learning Systems* 30 (11) (2019) 3212–3232, <https://doi.org/10.1109/TNNLS.2018.2876865>.
 - [41] N. Zhang, J. Donahue, R. Girshick, T. Darrell, Part-based R-CNNs for fine-grained category detection, in: *European Conference on Computer Vision*, 2014, pp. 834–849, https://doi.org/10.1007/978-3-319-10590-1_54.
 - [42] C.R. Qi, W. Liu, C. Wu, H. Su, L.J. Guibas, Frustum pointnets for 3d object detection from rgb-d data, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 918–927, <https://doi.org/10.1109/CVPR.2018.00102>.
 - [43] S. Arabi, A. Haghighat, A. Sharma, A deep-learning-based computer vision solution for construction vehicle detection, *Comput. -Aided Civ. Infrastruct. Eng.* 35 (7) (2020) 753–767, <https://doi.org/10.1111/mice.12530>.
 - [44] H. Son, H. Choi, H. Seong, C. Kim, Detection of construction workers under varying poses and changing background in image sequences via very deep residual networks, *Autom. Constr.* 99 (2019) 27–38, <https://doi.org/10.1016/j.autcon.2018.11.033>.
 - [45] W. Fang, L. Ding, B. Zhong, P.E. Love, H. Luo, Automated detection of workers and heavy equipment on construction sites: a convolutional neural network approach, *Adv. Eng. Inform.* 37 (2018) 139–149, <https://doi.org/10.1016/j.aei.2018.05.003>.
 - [46] A. Braun, A. Borrmann, Combining inverse photogrammetry and BIM for automated labeling of construction site images for machine learning, *Autom. Constr.* 106 (2019), 102879, <https://doi.org/10.1016/j.autcon.2019.102879>.
 - [47] A. Adán, B. Quintana, S.A. Prieto, F. Bosché, Scan-to-BIM for ‘secondary’ building components, *Adv. Eng. Inform.* 37 (2018) 119–138, <https://doi.org/10.1016/j.aei.2018.05.001>.
 - [48] J. Long, E. Shelhamer, T. Darrell, Fully convolutional networks for semantic segmentation, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2015, pp. 3431–3440, <https://doi.org/10.1109/CVPR.2015.7298965>.
 - [49] T. Czerniawski, F. Leite, Automated segmentation of RGB-D images into a comprehensive set of building components using deep learning, *Adv. Eng. Inform.* 45 (2020), 101131, <https://doi.org/10.1016/j.aei.2020.101131>.
 - [50] A. Dimitrov, M. Golparvar-Fard, Vision-based material recognition for automated monitoring of construction progress and generating building information modeling from unordered site image collections, *Adv. Eng. Inform.* 28 (1) (2014) 37–49, <https://doi.org/10.1016/j.aei.2013.11.002>.
 - [51] C.V. Dung, Autonomous concrete crack detection using deep fully convolutional neural network, *Autom. Constr.* 99 (2019) 52–58, <https://doi.org/10.1016/j.autcon.2018.11.028>.
 - [52] A. Zhang, K.C. Wang, B. Li, E. Yang, X. Dai, Y. Peng, Y. Fei, Y. Liu, J.Q. Li, C. Chen, Automated pixel-level pavement crack detection on 3D asphalt surfaces using a deep-learning network, *Comput. -Aided Civ. Infrastruct. Eng.* 32 (10) (2017) 805–819, <https://doi.org/10.1111/mice.12297>.
 - [53] K. He, G. Gkioxari, P. Dollár, R. Girshick, Mask R-CNN, in: *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 2961–2969, <https://doi.org/10.1109/ICCV.2017.322>.

- [54] H. Ying, S. Lee, A mask R-CNN based approach to automatically construct as-is IFC BIM objects from digital images, in: Proceedings of 36th International Symposium on Automation and Robotics in Construction, 2019, pp. 764–771, <https://doi.org/10.22260/ISARC2019/0103>.
- [55] S. Naha, Q. Xiao, P. Banik, M.A. Reza, D.J. Crandall, Pose-guided knowledge transfer for object part segmentation, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops, 2020, pp. 906–907, <https://doi.org/10.1109/CVPRW50498.2020.00461>.
- [56] A. Kirillov, K. He, R. Girshick, C. Rother, P. Dollár, Panoptic segmentation, in: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2019, pp. 9404–9413, <https://doi.org/10.1109/CVPR.2019.00963>.
- [57] W.T. Calderon, D. Roberts, M. Golparvar-Fard, Synthesizing pose sequences from 3D assets for vision-based activity analysis, J. Comput. Civ. Eng. 35 (1) (2021) 04020052, [https://doi.org/10.1061/\(ASCE\)CP.1943-5487.0000937](https://doi.org/10.1061/(ASCE)CP.1943-5487.0000937).
- [58] B. Mahmood, S. Han, J. Seo, Implementation experiments on convolutional neural network training using synthetic images for 3D pose estimation of an excavator on real images, Autom. Constr. 133 (2022), 103996, <https://doi.org/10.1016/j.autcon.2021.103996>.
- [59] D. Dwibedi, I. Misra, M. Hebert, Cut, paste and learn: Surprisingly easy synthesis for instance detection, in: Proceedings of the IEEE international Conference on Computer Vision, 2017, pp. 1301–1310, <https://doi.org/10.1109/ICCV.2017.146>.
- [60] M.M. Soltani, Z. Zhu, A. Hammad, Automated annotation for visual recognition of construction resources using synthetic images, Autom. Constr. 62 (2016) 14–23, <https://doi.org/10.1016/j.autcon.2015.10.002>.
- [61] P. Isola, J.Y. Zhu, T. Zhou, A.A. Efros, Image-to-image translation with conditional adversarial networks, Proc. IEEE Conf. Comput. Vis. Pattern Recognit. 2017 (2017) 1125–1134, <https://doi.org/10.1109/CVPR.2017.632>.
- [62] T. Salimans, I. Goodfellow, W. Zaremba, V. Cheung, A. Radford, X. Chen, Improved techniques for training GANs, Advances in Neural Information Processing Systems (2016) 2234–2242, <https://doi.org/10.1109/WACV48630.2021.00134>.
- [63] L. Mescheder, A. Geiger, S. Nowozin, Which training methods for GANs do actually converge?, in: International Conference on Machine Learning, 2018, pp. 3481–3490, <https://doi.org/10.1007/978-3-642-35740-4134>.
- [64] H. Ying, S. Lee, A rule-based system to automatically validate IFC second-level space boundaries for building energy analysis, Autom. Constr. 127 (2021), 103724, <https://doi.org/10.1016/j.autcon.2021.103724>.
- [65] IFC Engine, <https://rdf.bg/product-list/ifc-engine/> (Last Accessed: February 19th, 2023).
- [66] xbm toolkit, <https://docs.xbm.net/> (Last Accessed: February 19th, 2023).
- [67] H. Ying, S. Lee, Generating second-level space boundaries from large-scale IFC-compliant building information models using multiple geometry representations, Autom. Constr. 126 (2021), 103659, <https://doi.org/10.1016/j.autcon.2021.103659>.
- [68] J. Canny, A computational approach to edge detection, IEEE Trans. Pattern Anal. Mach. Intell. 6 (1986) 679–698, <https://doi.org/10.1109/TPAMI.1986.4767851>.
- [69] S. Daum, A. Borrmann, Processing of topological BIM queries using boundary representation based methods, Adv. Eng. Inform. 28 (4) (2014) 272–286, <https://doi.org/10.1016/j.aei.2014.06.001>.
- [70] H. Ying, H. Zhou, A. Degani, R. Sacks, A two-stage recursive ray tracing algorithm to automatically identify external building objects in building information models, Comput. -Aided Civ. Infrastruct. Eng. 37 (8) (2022) 991–1009, <https://doi.org/10.1111/mice.12776>.
- [71] A. Borrmann, S. Schraufstetter, E. Rank, Implementing metric operators of a spatial query language for 3D building models: octree and B-rep approaches, J. Comput. Civ. Eng. 23 (1) (2009) 34–46, [https://doi.org/10.1061/\(ASCE\)0887-3801\(2009\)23:1\(34\)](https://doi.org/10.1061/(ASCE)0887-3801(2009)23:1(34)).
- [72] H. Ying, S. Lee, Automatic detection of geometric errors in space boundaries of IFC-BIM models using Monte Carlo ray tracing approach, J. Comput. Civ. Eng. 34 (2) (2020) 04019056, [https://doi.org/10.1061/\(ASCE\)CP.1943-5487.0000878](https://doi.org/10.1061/(ASCE)CP.1943-5487.0000878).
- [73] J.D. MacDonald, K.S. Booth, Heuristics for ray tracing using space subdivision, Vis. Comput. 6 (3) (1990) 153–166, <https://doi.org/10.1007/BF01911006>.
- [74] A. Williams, S. Barrus, R.K. Morley, P. Shirley, An efficient and robust ray-box intersection algorithm, J. Graphics Tools 10 (1) (2005) 49–54, <https://doi.org/10.1145/1198555.1198748>.
- [75] T. Möller, B. Trumbore, Fast, minimum storage ray/triangle intersection, J. Graphics Tools 2 (1) (1997) 21–28, <https://doi.org/10.1080/10867651.1997.10487468>.
- [76] Q.Y. Zhou, J. Park, V. Koltun, Open3D: A modern library for 3D data processing, arXiv preprint (2018), <https://doi.org/10.48550/arXiv.1801.09847>.
- [77] Solibri, <https://www.solibri.com/solibri-office> (Last Accessed: February 19th, 2023).
- [78] G.A. Miller, WordNet, A Lexical Database for English, Communications of the ACM 38 (11) (1995) pp.39–41, <https://doi.org/10.1145/219717.219748>.
- [79] P. Pauwels, W. Terkaj, EXPRESS to OWL for construction industry: Towards a recommendable and usable ifcOWL ontology, Autom. Constr. 63 (2016) pp.100–133, <https://doi.org/10.1016/j.autcon.2015.12.003>.
- [80] Brick, <https://brickschema.org/> (Last Accessed: February 10th, 2023).
- [81] OmniClass, <https://www.csiresources.org/standards/omniclass> (Last Accessed: February 10th, 2023).
- [82] Uniclass, <https://www.thenbs.com/our-tools/uniclass> (Last Accessed: February 10th, 2023).
- [83] L. Ma, R. Sacks, U. Kattel, T. Bloch, 3D object classification using geometric features and pairwise relationships, Comput. -Aided Civ. Infrastruct. Eng. 33 (2) (2018) 152–164, <https://doi.org/10.1111/mice.12336>.
- [84] J. Wu, T. Akanbi, J. Zhang, Constructing invariant signatures for AEC objects to support BIM-based analysis automation through object classification, J. Comput. Civ. Eng. 36 (4) (2022) 04022008, [https://doi.org/10.1061/\(ASCE\)CP.1943-5487.0001012](https://doi.org/10.1061/(ASCE)CP.1943-5487.0001012).
- [85] C. Emunds, N. Pauen, V. Richter, J. Frisch, C. van Treeck, SpaRSE-BIM: classification of IFC-based geometry via sparse convolutional neural networks, Adv. Eng. Inform. 53 (2022), 101641, <https://doi.org/10.1016/j.aei.2022.101641>.
- [86] Blender, <https://www.blender.org/> (Last Accessed: February 10th, 2023).
- [87] Unity, <https://unity.com/> (Last Accessed: February 10th, 2023).
- [88] COCO, <https://cocodataset.org/#detection-eval> (Last Accessed: May 17th, 2023).